

AWS 入門

aoki-taquan

2026 年 04 月 21 日

目次

1	はじめに	9
1.1	AWS とは	9
1.2	クラウドの基本概念	9
1.2.1	クラウドコンピューティングの定義	9
1.2.2	サービスモデル	9
1.3	責任共有モデル	10
1.4	グローバルインフラストラクチャ	10
1.4.1	リージョンとアベイラビリティゾーン	10
1.4.2	リージョン選択の考え方	10
1.4.3	リージョンをまたがないリソース	10
1.5	料金モデル	11
1.5.1	従量課金制	11
1.5.2	主な料金構成要素	11
1.5.3	割引の仕組み	11
1.5.4	無料利用枠 (Free Tier)	11
1.6	操作手段	11
1.7	本書の対象読者	12
1.8	前提知識	12
1.9	本書の進め方	12
2	アカウント開設と初期設定	13
2.1	AWS アカウントの開設	13
2.1.1	開設に必要なもの	13
2.1.2	開設手順の概要	13
2.2	ルートユーザーの保護	13
2.2.1	ルートユーザーに必須の設定	13
2.2.2	ルートユーザーでしかできない作業	14
2.3	請求アラートの設定	14
2.3.1	IAM ユーザーによる請求情報の閲覧許可	14
2.3.2	AWS Budgets で予算アラートを作る	14
2.3.3	無料利用枠アラート	14
2.4	IAM Identity Center (旧 AWS SSO)	15
2.4.1	従来の IAM ユーザー方式との違い	15
2.4.2	基本的なセットアップ手順	15
2.4.3	CLI からのログイン	15
2.5	作業用 IAM ユーザーの作成 (Identity Center を使わない場合)	16
2.6	リージョンの選択	16
2.7	AWS CLI のセットアップ	16
2.7.1	インストール	16
2.7.2	認証情報の設定	16

2.7.3	動作確認	17
2.8	初期設定のチェックリスト	17
3	IAM (認証・認可)	18
3.1	IAM の主要コンポーネント	18
3.1.1	4つの基本要素	18
3.1.2	認証と認可の流れ	18
3.2	IAM ポリシーの構造	18
3.2.1	主要フィールド	19
3.2.2	ARN (Amazon Resource Name)	19
3.2.3	条件 (Condition)	19
3.3	ポリシーの種類	20
3.3.1	管理ポリシーとインラインポリシー	20
3.3.2	アイデンティティベースとリソースベース	20
3.3.3	SCP (サービスコントロールポリシー)	20
3.4	ロールと一時認証情報	20
3.4.1	なぜロールが必要か	20
3.4.2	ロールの使われ方	20
3.4.3	信頼ポリシーとアクセスポリシー	20
3.4.4	GitHub Actions から OIDC で引き受ける例	21
3.5	ベストプラクティス	21
3.5.1	最小権限の原則	21
3.5.2	権限境界 (Permissions Boundary)	22
3.5.3	実装上の推奨	22
3.5.4	アクセスキーを CI に置かない	22
3.6	実習: 権限不足を体験する	22
3.7	よく使う AWS 管理ポリシー	22
4	VPC (ネットワーク)	24
4.1	VPC の基本構造	24
4.1.1	階層	24
4.1.2	CIDR ブロックの設計	24
4.1.3	デフォルト VPC	24
4.2	サブネットとルートテーブル	25
4.2.1	パブリックサブネットとプライベートサブネット	25
4.2.2	ルートテーブルの役割	25
4.3	インターネット接続	25
4.3.1	Internet Gateway (IGW)	25
4.3.2	NAT Gateway	25
4.3.3	外向き専用 Internet Gateway (Egress-Only IGW)	26
4.3.4	パブリック IP	26
4.4	セキュリティグループとネットワーク ACL	26
4.4.1	セキュリティグループ (SG)	26
4.4.2	SG 間参照	26

4.4.3	ネットワーク ACL (NACL)	27
4.5	VPC エンドポイント	27
4.5.1	ゲートウェイ型エンドポイント	27
4.5.2	インタフェース型エンドポイント	27
4.6	VPC 間・オンプレ接続	27
4.6.1	VPC ピアリング	27
4.6.2	Transit Gateway	27
4.6.3	Site-to-Site VPN	28
4.6.4	Direct Connect	28
4.7	DNS と DHCP	28
4.7.1	VPC のデフォルト DNS	28
4.7.2	DNS ホスト名と DNS 解決	28
4.7.3	プライベートホストゾーン	28
4.8	実践：小規模な Web 構成の VPC 設計	28
4.9	VPC 作成時の注意点	29
5	EC2 (仮想マシン)	30
5.1	EC2 の基本概念	30
5.1.1	インスタンス	30
5.1.2	料金体系	30
5.2	AMI (Amazon Machine Image)	30
5.2.1	主な AMI の出所	30
5.2.2	Amazon Linux 2023	31
5.2.3	カスタム AMI の作り方	31
5.3	インスタンスタイプ	31
5.3.1	主要ファミリー	31
5.3.2	世代と CPU アーキテクチャ	31
5.3.3	t ファミリーのクレジット仕様	32
5.4	ストレージ (EBS / インスタンスストア)	32
5.4.1	EBS (Elastic Block Store)	32
5.4.2	インスタンスストア	32
5.4.3	ルートボリュームの扱い	32
5.5	ネットワーク関連	33
5.5.1	キーペアと SSH	33
5.5.2	SSM Session Manager (推奨)	33
5.5.3	Elastic IP	33
5.6	Auto Scaling とロードバランサ	34
5.6.1	ロードバランサ (ELB)	34
5.6.2	ターゲットグループとヘルスチェック	34
5.6.3	Auto Scaling グループ (ASG)	34
5.7	ユーザーデータと初期化	34
5.8	実習：ALB + EC2 の最小構成	35
5.9	トラブルシューティング	35

6	S3 (オブジェクトストレージ)	36
6.1	S3 の基本概念	36
6.1.1	バケットとオブジェクト	36
6.1.2	耐久性と可用性	36
6.1.3	バケット名の制約	36
6.2	ストレージクラス	36
6.2.1	ライフサイクルルール	37
6.3	操作の基本	37
6.3.1	CLI によるアップロード／ダウンロード	37
6.3.2	プリサインド URL	38
6.4	アクセス制御	38
6.4.1	階層	38
6.4.2	推奨デフォルト	38
6.4.3	バケットポリシーの例	39
6.4.4	ブロックパブリックアクセスのバイパス	39
6.5	バージョニングとオブジェクトロック	39
6.5.1	バージョニング	39
6.5.2	オブジェクトロック (WORM)	39
6.6	静的ウェブサイトホスティング	40
6.7	イベント通知	40
6.8	クロスリージョンレプリケーションと S3 Replication Time Control	40
6.9	コスト最適化のポイント	41
6.10	S3 のよくあるつまずき	41
7	データベース	42
7.1	サービス選択の早見表	42
7.2	RDS (Relational Database Service)	42
7.2.1	サポートエンジン	42
7.2.2	インスタンスクラスとストレージ	42
7.2.3	高可用性：Multi-AZ 配置	43
7.2.4	リードレプリカ	43
7.2.5	バックアップと復元	43
7.2.6	パラメータグループとオプショングループ	43
7.3	Aurora	43
7.3.1	Aurora の特徴	43
7.3.2	Aurora Serverless v2	43
7.3.3	Aurora Global Database	44
7.4	DynamoDB	44
7.4.1	基本構造	44
7.4.2	料金モード	44
7.4.3	Single Table Design	44
7.4.4	DynamoDB Streams	44
7.4.5	Global Tables	44

7.5	ElastiCache	44
7.5.1	選び方	44
7.5.2	典型的なユースケース	45
7.5.3	クラスタモード	45
7.6	DB 選定のガイドライン	45
7.6.1	「まず RDBMS」で始める	45
7.6.2	マネージドかセルフホストか	45
7.6.3	商用ライセンスの扱い	45
7.7	DB 周りのネットワーク	45
7.8	バックアップとリカバリのベストプラクティス	46
7.9	コスト最適化のポイント	46
8	コンテナとサーバーレス	47
8.1	コンピュータ選択のスペクトラム	47
8.2	Lambda (サーバーレス関数)	47
8.2.1	実行モデル	47
8.2.2	料金モデル	47
8.2.3	コールドスタート	47
8.2.4	シンプルな Lambda の例 (Python)	48
8.2.5	Lambda のユースケース	48
8.2.6	制限	48
8.3	API Gateway	48
8.3.1	種類	48
8.3.2	Lambda Proxy 連携	49
8.3.3	認証／認可	49
8.4	ECS / EKS / Fargate	49
8.4.1	ECS (Elastic Container Service)	49
8.4.2	EKS (Elastic Kubernetes Service)	49
8.4.3	Fargate	49
8.4.4	ECR (Elastic Container Registry)	50
8.4.5	ECS タスク定義の例	50
8.5	App Runner / Elastic Beanstalk / Amplify	50
8.5.1	App Runner	51
8.5.2	Elastic Beanstalk	51
8.5.3	Amplify	51
8.6	イベント駆動・非同期アーキテクチャ	51
8.6.1	SQS + Lambda の基本パターン	51
8.6.2	Step Functions	52
8.7	サーバーレス／コンテナの選定指針	52
8.8	参考アーキテクチャ：サーバーレス Web アプリ	52
9	運用と監視	53
9.1	CloudWatch	53
9.1.1	CloudWatch Metrics	53

9.1.2	アラーム	53
9.1.3	CloudWatch Logs	53
9.1.4	Logs Insights	54
9.1.5	ダッシュボード	54
9.1.6	CloudWatch Alarms の通知をメール以外に	54
9.2	CloudTrail	54
9.2.1	イベント種別	54
9.2.2	証跡 (Trail)	54
9.2.3	主要ログフィールド	54
9.3	AWS Config	55
9.3.1	典型的な利用シーン	55
9.4	Systems Manager (SSM)	55
9.4.1	主要機能	55
9.4.2	Session Manager のメリット	55
9.4.3	Parameter Store	56
9.4.4	Patch Manager	56
9.5	監視・運用の全体像	56
9.6	タグ戦略	57
9.7	アラート疲れを防ぐ	57
9.8	運用のベストプラクティス	57
10	セキュリティとコスト管理	58
10.1	セキュリティの全体像	58
10.2	KMS (Key Management Service)	58
10.2.1	キーの種類	58
10.2.2	キーポリシーと IAM	58
10.2.3	エンベロープ暗号化	58
10.3	Secrets Manager と Parameter Store	59
10.3.1	使い分け	59
10.3.2	DB パスワードのローテーション	59
10.4	WAF と Shield	59
10.4.1	AWS WAF	59
10.4.2	Shield / Shield Advanced	59
10.5	GuardDuty / Inspector / Macie / Security Hub	60
10.5.1	GuardDuty	60
10.5.2	Inspector	60
10.5.3	Macie	60
10.5.4	Security Hub	60
10.6	IAM Access Analyzer	60
10.7	AWS Organizations と SCP	60
10.7.1	管理構造	61
10.7.2	SCP (サービスコントロールポリシー)	61
10.7.3	Control Tower	61

10.8	コスト管理サービス	62
10.8.1	AWS Budgets	62
10.8.2	Cost Explorer	62
10.8.3	Cost and Usage Report (CUR)	62
10.8.4	Savings Plans と Reserved Instance	62
10.8.5	Spot Instance	62
10.8.6	コスト最適化のチェックリスト	62
10.9	セキュリティ+コストで最低限やること	63
11	IaC と運用 Tips	64
11.1	なぜ IaC が必要か	64
11.2	CloudFormation	64
11.2.1	基本概念	64
11.2.2	最小テンプレート例	64
11.2.3	適用	65
11.2.4	特徴	65
11.2.5	SAM (Serverless Application Model)	65
11.3	AWS CDK (Cloud Development Kit)	66
11.3.1	特徴	66
11.3.2	TypeScript での例	66
11.3.3	デプロイフロー	66
11.3.4	CloudFormation との使い分け	67
11.4	Terraform	67
11.4.1	特徴	67
11.4.2	例	67
11.4.3	コマンド	68
11.4.4	state 管理の注意	68
11.5	IaC ツール選びのガイドライン	68
11.6	運用 Tips とハマりどころ	68
11.6.1	リージョン設定の一貫性	68
11.6.2	名前衝突の回避	69
11.6.3	削除できないリソース	69
11.6.4	マルチアカウント設計	69
11.6.5	CI/CD での認証	69
11.6.6	よくある事故と回避	69
11.6.7	リソース棚卸しの習慣	70
11.7	さらに学ぶために	70

1 はじめに

1.1 AWS とは

Amazon Web Services (AWS) は、Amazon.com が提供する総合クラウドコンピューティングサービスである。2006 年に S3 と EC2 の提供を開始して以来、計算・ストレージ・ネットワーク・データベース・機械学習・IoT・セキュリティなど 200 を超えるサービスを展開しており、パブリッククラウド市場で最大級のシェアを持つ。

従来、サーバーを運用するには物理マシンを購入し、データセンターに設置し、OS やミドルウェアを構築する必要があった。AWS はこれらの IT リソースをインターネット経由で必要なときに必要なだけ利用できる形で提供する。初期投資なしに、数分でサーバーを起動し、不要になれば即座に破棄できる。

1.2 クラウドの基本概念

1.2.1 クラウドコンピューティングの定義

米国 NIST（国立標準技術研究所）は、クラウドコンピューティングを次の 5 つの特徴で定義している。

- ・ **オンデマンド・セルフサービス**：人手を介さず必要なときに利用開始できる
- ・ **幅広いネットワークアクセス**：インターネット経由でどこからでもアクセスできる
- ・ **リソースの共用**：複数のテナントで物理リソースを共有する
- ・ **スピーディな拡張性**：需要に応じて自動でリソースを増減できる
- ・ **サービスの計測可能性**：利用量を計測し、従量課金が可能である

1.2.2 サービスモデル

クラウドサービスは、抽象度によって以下のように分類される。

モデル	提供範囲	AWS の例
IaaS	仮想マシン・ストレージ・ネットワーク	EC2、VPC、EBS
PaaS	アプリ実行環境	Elastic Beanstalk、App Runner
SaaS	アプリケーション	WorkMail、Chime
FaaS Serverless	/ 関数単位の実行環境	Lambda

AWS は IaaS から SaaS まで幅広く提供しているため、要件に合わせて組み合わせられる。

1.3 責任共有モデル

AWS のセキュリティ責任は、AWS と利用者の間で分担される。これを **責任共有モデル (Shared Responsibility Model)** と呼ぶ。

担当	責任範囲
AWS	物理施設、ハードウェア、仮想化基盤、マネージドサービス内部の運用（「クラウドのセキュリティ」）
利用者	OS 以上の設定、データ、IAM 権限、ネットワーク設定、アプリケーション（「クラウド内のセキュリティ」）

EC2 のような IaaS では OS パッチ適用まで利用者の責任だが、S3 や RDS のようなマネージドサービスではミドルウェア以下は AWS が責任を負う。サービスごとに境界が異なる点に注意する。

1.4 グローバルインフラストラクチャ

1.4.1 リージョンとアベイラビリティゾーン

AWS のインフラは世界中に分散配置されている。

- ・ **リージョン (Region)** : 地理的に独立したデータセンター群の集まり。互いに物理的・ネットワーク的に分離されている
- ・ **アベイラビリティゾーン (AZ)** : 1つのリージョン内で電源・ネットワーク・冷却が独立した複数のデータセンター群。各リージョンには通常 3 つ以上の AZ がある
- ・ **エッジロケーション** : CloudFront や Route 53 等で使われる配信拠点。リージョンよりはるかに多い

日本には **東京 (ap-northeast-1)** と **大阪 (ap-northeast-3)** の 2 つのリージョンがある。低レイテンシが必要な場合は東京を、災害対策のマルチリージョン構成では東京+大阪を選ぶのが一般的である。

1.4.2 リージョン選択の考え方

- ・ **レイテンシ** : ユーザーに近いリージョンを選ぶ
- ・ **法規制・データ主権** : データの保管場所に制約がある場合は国内リージョンを選ぶ
- ・ **料金** : リージョンごとに料金が異なる（東京はバージニア北部より高めの傾向）
- ・ **サービス可用性** : 新サービスはバージニア北部 (us-east-1) から提供開始されることが多い

1.4.3 リージョンをまたがないリソース

ほとんどの AWS リソースはリージョン固有である。東京リージョンで作成した EC2 インスタンスは、大阪リージョンのコンソールには表示されない。一方で IAM、Route 53、CloudFront、S3 のバケット名空間などは **グローバルサービス** として扱われる。

1.5 料金モデル

1.5.1 従量課金制

AWS は原則として使った分だけ支払う **従量課金制 (Pay-as-you-go)** である。固定の月額契約はなく、利用時間・転送量・リクエスト数などに応じて課金される。

1.5.2 主な料金構成要素

項目	例
計算	EC2 インスタンスの稼働時間、Lambda の実行時間
ストレージ	EBS ボリュームの容量、S3 の保存容量
データ転送	インターネット向け下り（アウトバウンド）転送量
リクエスト	S3 の PUT/GET、API Gateway のリクエスト数
付随サービス	NAT Gateway の時間課金、ロードバランサの稼働時間

初心者がハマりやすいのは、**インスタンスを止め忘れた月末の請求と NAT Gateway を立てっぱなしにした固定費** である。個人利用では特に注意する。

1.5.3 割引の仕組み

長期利用を前提とする場合、以下の割引が使える。

- **Savings Plans / Reserved Instance** : 1 年または 3 年のコミットで最大約 70% 割引
- **Spot Instance** : 中断許容のワークロード向けに最大 90% 割引
- **ボリュームディスカウント** : S3 や データ転送の利用量階段制

1.5.4 無料利用枠 (Free Tier)

AWS には新規アカウント向けに **Free Tier** が用意されている。

- **12 か月間無料** : EC2 `t2.micro` または `t3.micro` を月 750 時間など
- **常時無料** : Lambda の月 100 万リクエスト、DynamoDB の 25GB など
- **トライアル** : 一部サービスで 30 日無料など

Free Tier は便利だが、上限超過に気づきにくい。後述する予算アラートを必ず設定する。

1.6 操作手段

AWS は同じ機能を複数の手段で操作できる。

- **マネジメントコンソール** : ブラウザベースの GUI。学習や単発作業に向く
- **AWS CLI** : コマンドラインから操作する。スクリプト化に向く
- **AWS SDK** : 各言語 (Python の `boto3` など) からプログラマ的に操作
- **CloudShell** : ブラウザ上のシェル環境。CLI と SDK がプリインストール済み
- **IaC** : CloudFormation / CDK / Terraform で宣言的に管理

本書では主にマネジメントコンソールと AWS CLI を用いて解説する。

1.7 本書の対象読者

- これから AWS を学び始める開発者・インフラエンジニア
- 自宅ラボや個人プロジェクトで AWS を試したい方
- クラウド全般の基礎を押さえたい方
- AWS 認定資格（CLF / SAA）の前段として概観をつかみたい方

1.8 前提知識

本書では以下の知識があることを前提とする。

- Linux の基本操作（SSH 接続、シェルコマンド）
- TCP/IP の基礎（IP アドレス、サブネット、ポート）
- Web アプリケーションの基本構成（クライアント／サーバ、HTTP）

上記が不安な場合でも、必要な背景は随所で補足する。

1.9 本書の進め方

本書は、まず AWS のアカウントを開設し（2 章）、最小限の安全設定を行ったうえで、**IAM → VPC → EC2 → S3** という基礎 4 サービス（3～6 章）を順に理解していく構成である。7 章以降はデータベース、コンテナ・サーバーレス、運用・監視、セキュリティ・コスト管理、IaC と応用テーマを扱う。

手を動かして試す際は、必ず 2 章の予算アラートを先に設定し、作業後はリソースを削除する習慣を身につけることを強く推奨する。

2 アカウント開設と初期設定

AWS を利用する前に必要な、アカウントの作成と、事故を防ぐための初期設定について解説する。この章の作業は必ず最初に行うこと。数千円単位の請求事故を防ぐ基本動作である。

2.1 AWS アカウントの開設

2.1.1 開設に必要なもの

- ・メールアドレス（他の AWS アカウントで未使用のもの）
- ・クレジットカード（または一部デビットカード）
- ・電話番号（SMS または自動音声による本人確認）
- ・住所

メールアドレスは AWS アカウントのルートユーザーに紐づく重要な識別子である。個人名の Gmail ではなく、長期的に維持できる共用アドレス（`aws-root+company@example.com` のような形式）を使うのが望ましい。

2.1.2 開設手順の概要

1. 公式サイトから **AWS アカウントを作成** を選択
2. メールアドレス、パスワード、アカウント名を入力
3. 連絡先情報（個人 / ビジネス）、住所を入力
4. クレジットカードを登録（即時 1 ドル程度のオーソリがかかる）
5. 電話番号で本人確認（SMS または音声）
6. サポートプランを選択（最初は **ベーシック（無料）** で十分）

数分でアカウントが有効になり、マネジメントコンソールにログインできるようになる。

2.2 ルートユーザーの保護

AWS アカウント作成時のメールアドレスでログインするユーザーを **ルートユーザー** と呼ぶ。ルートユーザーはアカウント解約や支払い方法変更を含む全権限を持つため、**日常作業では絶対に使わない**。

2.2.1 ルートユーザーに必須の設定

1. **強固なパスワードを設定**：パスワードマネージャで生成した長いランダム文字列を使う
2. **MFA（多要素認証）を有効化**：仮想 MFA（Google Authenticator、1Password 等）またはハードウェアキー（YubiKey 等）
3. **アクセスキーを作らない**：ルートユーザーのアクセスキーは使わない、存在すれば削除する
4. **通知先メールを安全に管理**：乗っ取り時の復旧経路になる

MFA の設定は **IAM コンソール** の「セキュリティ認証情報」から行う。仮想 MFA アプリで QR コードを読み取り、6 桁のコードを 2 回入力すれば有効化される。

2.2.2 ルートユーザーでしかできない作業

以下はルートユーザーでしか実行できない。日常的には発生しないが、どのような操作かを把握しておく。

- アカウント名・ルートメールアドレスの変更
- 支払い方法の変更、税務情報の設定
- AWS サポートプランの解約
- AWS アカウント自体の解約
- S3 バケットのデフォルト暗号化ポリシーの一部設定

2.3 請求アラートの設定

AWS で最初に必ず設定すべきは **予算アラート** である。使いすぎに気づく唯一の自動経路である。

2.3.1 IAM ユーザーによる請求情報の閲覧許可

初期状態では、ルートユーザー以外は請求情報にアクセスできない。以下の手順で IAM ユーザーからも見られるようにする。

1. ルートユーザーで **アカウント** ページを開く
2. **IAM ユーザー／ロールによる請求情報へのアクセス** を有効化
3. この設定はアカウント全体に適用される

2.3.2 AWS Budgets で予算アラートを作る

Budgets は予算超過を検知してメール通知するサービスである。

1. コンソールから **Billing** → **Budgets** を開く
2. **予算を作成** をクリック
3. **コスト予算** を選択
4. 期間を **月次**、金額を例えば **10 USD** と設定
5. アラートしきい値を **実際の 80%** と **予測の 100%** で 2 段階構えにする
6. 通知先メールアドレスを指定

これで毎月の使用額が設定額に近づくと通知が届くようになる。個人利用では必ず最初にやる。

2.3.3 無料利用枠アラート

Free Tier の消化率でアラートを出す設定も併用できる。

- **Billing** → **Billing Preferences**
- **Free Tier usage alerts** を有効化してメールアドレスを登録

85% 消化時点で自動通知される。Free Tier はサービスごとに条件が細かいため、この機能の利用を推奨する。

2.4 IAM Identity Center (旧 AWS SSO)

IAM Identity Center は、AWS の公式なシングルサインオン／アイデンティティ基盤である。個人利用でも小規模利用でも、長期運用を見据えるなら **最初から Identity Center で運用する** のが現代的な選択である。

2.4.1 従来の IAM ユーザー方式との違い

項目	IAM ユーザー	IAM Identity Center
作成単位	AWS アカウント内	組織全体で一元管理
認証	長期パスワード／アクセスキー	SSO、短期認証情報
MFA	任意で設定	標準で設定
マルチアカウント	各アカウントで別ユーザー	1 ユーザーで複数アカウントにアクセス
CLI アクセス	長期アクセスキー	<code>aws sso login</code> で一時認証情報

2.4.2 基本的なセットアップ手順

1. コンソールで **IAM Identity Center** を開く
2. **有効にする** をクリック（AWS Organizations が自動作成される）
3. **ユーザー** を追加し、メールアドレスを登録
4. **許可セット** を作成（例： `AdministratorAccess`、 `ReadOnlyAccess` ）
5. **AWS アカウント** 画面で、ユーザー／グループに許可セットを割り当てる
6. ユーザーに届いた招待メールから初回パスワードと MFA を設定

以降、コンソールへのアクセスは Identity Center のログインポータル URL（例：`https://d-xxxxxxxxx.awsapps.com/start`）を経由する。

2.4.3 CLI からのログイン

CLI を Identity Center で使う設定は以下の通り。

```
aws configure sso
# SSO start URL: https://d-xxxxxxxxx.awsapps.com/start
# SSO region: ap-northeast-1
# ブラウザが開くのでログイン
# アカウント・ロールを選択
# プロファイル名を入力（例: dev）

aws sso login --profile dev
aws s3 ls --profile dev
```

短期認証情報（最大12時間）が自動発行され、長期アクセスキーをローカルに置く必要がなくなる。漏洩リスクが大幅に下がるため、可能ならこちらを使う。

2.5 作業用 IAM ユーザーの作成（Identity Center を使わない場合）

個人検証で Identity Center の設定が重いと感じる場合は、従来型の IAM ユーザーで始めてもよい。ただしセキュリティ上の妥協点であることは認識する。

1. IAM コンソール → ユーザー → ユーザーを作成
2. ユーザー名を入力（例： `your-name`）
3. コンソールアクセスを有効化 をチェック、パスワードを設定
4. 権限では 既存のポリシーを直接アタッチ → `AdministratorAccess` を選択（検証用途のみ）
5. 作成後、MFA を必ず有効化する

本番環境や他人と共有するアカウントでは `AdministratorAccess` を恒常的に付けず、Identity Center の一時権限を使うべきである。

2.6 リージョンの選択

コンソール右上のリージョン切替で、操作対象のリージョンを選ぶ。日本国内でのレイテンシ重視なら 東京（`ap-northeast-1`）が基本である。一度設定すると、そのリージョンのリソースだけが表示される 点に注意する。

- ・ グローバルサービス（IAM、Route 53、CloudFront、S3 の一部）はリージョン切替に関わらず 共通
- ・ 東京で作ったリソースは、うっかりバージニア北部を開くと「何も無い」ように見えるため混乱しやすい

2.7 AWS CLI のセットアップ

2.7.1 インストール

Linux / macOS では以下で導入できる。

```
# Linux (x86_64)
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscli.zip
unzip awscli.zip
sudo ./aws/install

# macOS (Homebrew)
brew install awscli

# 動作確認
aws --version
```

2.7.2 認証情報の設定

Identity Center を使う場合は `aws configure sso`（前述）を使う。アクセスキー方式の場合は以下。


```
aws configure
# AWS Access Key ID: ...
# AWS Secret Access Key: ...
# Default region name: ap-northeast-1
# Default output format: json
```

設定は `~/.aws/credentials` と `~/.aws/config` に保存される。`--profile` オプションで複数のアカウント／ロールを切り替えられる。

2.7.3 動作確認

```
# 現在の認証情報を確認
aws sts get-caller-identity

# S3 バケット一覧
aws s3 ls

# 東京リージョンの EC2 インスタンス一覧
aws ec2 describe-instances --region ap-northeast-1
```

2.8 初期設定のチェックリスト

作業を進める前に、以下がすべて完了していることを確認する。

- ☐ ルートユーザーに MFA を設定した
- ☐ ルートユーザーのアクセスキーは作成していない／削除した
- ☐ AWS Budgets で月次予算アラートを設定した
- ☐ Free Tier 使用量アラートを有効化した
- ☐ IAM Identity Center または作業用 IAM ユーザーを作成した
- ☐ 作業用ユーザーにも MFA を設定した
- ☐ 東京リージョンをデフォルトに設定した
- ☐ AWS CLI が動作し、`aws sts get-caller-identity` で自分のユーザーが確認できた

ここまで終われば、3 章以降の実習に進む準備が整っている。

3 IAM（認証・認可）

IAM（Identity and Access Management）は、「誰が（認証）」「何を（認可）」できるかを管理する AWS のセキュリティ基盤である。どのサービスを使うにしても IAM が土台になるため、最初に理解しておきたい。

3.1 IAM の主要コンポーネント

3.1.1 4 つの基本要素

要素	説明
ユーザー（User）	個人や単一のアプリに紐づく長期的なアイデンティティ
グループ（Group）	複数ユーザーをまとめる単位。グループにポリシーをアタッチできる
ロール（Role）	一時的に引き受けるアイデンティティ。EC2 や Lambda、他アカウントに付与する
ポリシー（Policy）	JSON で書かれた許可／拒否のルール

ポリシーを直接ユーザー／ロールにアタッチすることもできるし、グループ経由でアタッチすることもできる。

3.1.2 認証と認可の流れ

1. 認証（AuthN）：API コール時に、アクセスキー or 一時認証情報 or SSO によって「誰か」が特定される
2. 認可（AuthZ）：そのプリンシパルに付与されたポリシーと、対象リソース側のポリシーを評価して許可／拒否が決まる
3. 明示的 Deny はどのポリシーの Allow も上書きする

3.2 IAM ポリシーの構造

ポリシーは JSON で記述される。以下は S3 バケットへの読み取りを許可する最小例である。

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadMyBucket",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
```

```

    "arn:aws:s3::my-bucket",
    "arn:aws:s3::my-bucket/*"
  ]
}
]
}
```

3.2.1 主要フィールド

フィールド	意味
Version	必ず 2012-10-17 と書く（現行仕様の識別子）
Effect	Allow または Deny
Action	許可／拒否する API アクション。s3:* のようにワイルドカードも可
Resource	対象リソースの ARN（Amazon Resource Name）
Condition	IP、MFA、時刻など追加条件を指定
Principal	リソースベースポリシー（後述）で対象プリンシパルを指定

3.2.2 ARN（Amazon Resource Name）

AWS のあらゆるリソースは ARN と呼ばれる一意の識別子を持つ。

```
arn:aws:<service>:<region>:<account-id>:<resource>
```

例：

- arn:aws:s3::my-bucket （S3 はリージョンとアカウント ID が空）
- arn:aws:ec2:ap-northeast-1:123456789012:instance/i-0abcd1234efgh5678
- arn:aws:iam::123456789012:role/EC2-S3-ReadOnly

ポリシーの Resource 指定では ARN を使う。

3.2.3 条件（Condition）

アクセス元 IP、時刻、MFA の有無などで制御を追加できる。

```

{
  "Effect": "Allow",
  "Action": "s3:*",
  "Resource": "*",
  "Condition": {
    "Bool": { "aws:MultiFactorAuthPresent": "true" },
    "IpAddress": { "aws:SourceIp": "203.0.113.0/24" }
  }
}
```

MFA 必須や社内 IP 限定のアクセス制御はこの Condition で実現する。

3.3 ポリシーの種類

3.3.1 管理ポリシーとインラインポリシー

種類	説明
AWS 管理ポリシー	AWS が提供する既製ポリシー。AdministratorAccess、ReadOnlyAccess など
カスタマー管理ポリシー	利用者が作成する独立したポリシー。複数のユーザー／ロールにアタッチできる
インラインポリシー	ユーザー／ロールに直接埋め込む。1 対 1 で紐づくため他にアタッチできない

運用上は **カスタマー管理ポリシー** を作って再利用するのが望ましい。

3.3.2 アイデンティティベースとリソースベース

- ・ **アイデンティティベースポリシー**：IAM ユーザー／グループ／ロールにアタッチする
- ・ **リソースベースポリシー**：S3 バケット、SQS キュー、Lambda 関数などリソース側に定義する

S3 バケットポリシー、Lambda のリソースベースポリシー、KMS キーポリシーなどは後者の代表例である。

3.3.3 SCP（サービスコントロールポリシー）

AWS Organizations 配下で、アカウントそのものに対して使える機能を制限する。親アカウントから子アカウントへのガードレールとして機能する。例：「東京・大阪以外のリージョンでの API 実行を全面禁止」。

3.4 ロールと一時認証情報

3.4.1 なぜロールが必要か

EC2 で動くアプリが S3 にアクセスする場合、従来はアクセスキーをサーバに置いていた。これは漏洩リスクが高く、管理もしづらい。**IAM ロール** を使えば、インスタンスに一時認証情報が自動配布され、定期的にローテーションされる。

3.4.2 ロールの使われ方

- ・ **EC2 インスタンスプロファイル**：EC2 に付与する。インスタンス内からメタデータ経由で取得
- ・ **Lambda 実行ロール**：Lambda 関数が AWS API を呼ぶときの権限
- ・ **クロスアカウントロール**：別アカウントから sts:AssumeRole で引き受ける
- ・ **OIDC / SAML 連携ロール**：IdP（GitHub Actions など）からフェデレーションで引き受ける
- ・ **サービスリンクロール**：AWS 側が管理する特別なロール（ELB、ECS などが自動作成）

3.4.3 信頼ポリシーとアクセスポリシー

ロールには 2 種類のポリシーが紐づく。

- ・ 信頼ポリシー (Trust Policy) : 「誰がこのロールを引き受けられるか」
- ・ アクセスポリシー (Permissions Policy) : 「このロールが何をできるか」

EC2 がこのロールを引き受けられるようにする信頼ポリシーの例：

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "ec2.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
```

3.4.4 GitHub Actions から OIDC で引き受ける例

アクセスキーを発行せず、GitHub Actions から AWS を操作するためのモダンな構成。

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "Federated": "arn:aws:iam::123456789012:oidc-provider/
token.actions.githubusercontent.com"
    },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
      },
      "StringLike": {
        "token.actions.githubusercontent.com:sub": "repo:octo/app:*"
      }
    }
  }]
}
```

3.5 ベストプラクティス

3.5.1 最小権限の原則

必要最小限の権限だけを付与する。運用で権限不足が発生したら、CloudTrail のログや IAM Access Analyzer で実際に使われた API を確認し、必要なものを追加していく。

3.5.2 権限境界 (Permissions Boundary)

開発者が自分で IAM ロールを作れるが、特定の権限を超えないようにしたい場合に使う。IAM ロール／ユーザーの「権限の上限」を定義する。

3.5.3 実装上の推奨

- ・ ルートユーザーは使わない、MFA を必ず設定する
- ・ 長期アクセスキーを避ける、代わりにロール／Identity Center の一時認証情報を使う
- ・ グループ経由で権限付与、ユーザーに直接アタッチしない
- ・ カスタマー管理ポリシー を作り、複数ユーザーで共有する
- ・ アクセスキーを定期的にローテーション、未使用のキーは削除
- ・ IAM Access Analyzer で外部公開されたリソースを検出
- ・ CloudTrail で監査ログを取り続ける

3.5.4 アクセスキーを CI に置かない

GitHub Actions や GitLab CI など AWS を操作する場合は、長期アクセスキーをシークレットに保存するのではなく OIDC フェデレーションでロールを引き受ける。漏洩リスクがゼロに近づく。

3.6 実習：権限不足を体験する

IAM の感覚をつかむには、実際に権限を絞ってエラーを出す のが早い。

1. 管理者ユーザーとは別に、`ReadOnlyAccess` のみを付けた IAM ユーザー `demo-readonly` を作成
2. そのユーザーで AWS CLI の別プロファイルを設定
3. `aws ec2 describe-instances --profile demo-readonly` は成功する
4. `aws ec2 terminate-instances ... --profile demo-readonly` は `UnauthorizedOperation` で失敗する
5. エラー文言には必要なアクション名（例： `ec2:TerminateInstances` ）が含まれる

このフィードバックループを通じて、「エラー → 必要アクション特定 → ポリシー更新」の手順が身につく。

3.7 よく使う AWS 管理ポリシー

ポリシー名	用途
AdministratorAccess	全権限。検証・管理者用（本番で一般ユーザーに付けない）
ReadOnlyAccess	全サービスの読み取りのみ。監査・調査用
PowerUserAccess	IAM 以外ほぼ全権限。開発者用
AmazonS3FullAccess	S3 の全操作
AmazonEC2FullAccess	EC2 の全操作
IAMUserChangePassword	自分のパスワード変更を許可。全ユーザーに付けるとよい
AWSSupportAccess	サポートケースの作成・閲覧
Billing	請求情報の閲覧・管理

最初はこれらを組み合わせて始め、慣れてきたらカスタマー管理ポリシーで絞り込む。

4 VPC（ネットワーク）

VPC（Virtual Private Cloud）は、AWS 内に自分専用の仮想ネットワークを作るサービスである。EC2、RDS、Lambda（VPC モード）、EKS など、ほとんどの計算リソースは VPC の中に置かれる。AWS のネットワーク設計は VPC を中心に回る。

4.1 VPC の基本構造

4.1.1 階層

```
アカウント / リージョン
└─ VPC (例: 10.0.0.0/16)
    ├── AZ A
    │   ├── サブネット pub-a (10.0.1.0/24, パブリック)
    │   └── サブネット prv-a (10.0.11.0/24, プライベート)
    └── AZ C
        ├── サブネット pub-c (10.0.2.0/24, パブリック)
        └── サブネット prv-c (10.0.12.0/24, プライベート)
```

- **VPC**：リージョン単位。1つの CIDR ブロック（IPv4 アドレス範囲）を持つ
- **サブネット**：AZ 単位。VPC の CIDR を分割して割り当てる
- サブネットを跨ぐ通信はリージョン内では無料／低遅延

4.1.2 CIDR ブロックの設計

RFC1918 のプライベート IP レンジから選ぶのが基本。

- 10.0.0.0/8
- 172.16.0.0/12
- 192.168.0.0/16

AWS の VPC は /16 から /28 まで設定できる。将来の拡張を考えて /16 で広めに取り、サブネットは /24 単位で切ると扱いやすい。オンプレや他 VPC と接続する予定があるなら、CIDR 被りを避けるため事前に割り当てルールを決める。

4.1.3 デフォルト VPC

各リージョンには最初から **デフォルト VPC** が用意されている。172.31.0.0/16 のような CIDR で、各 AZ にパブリックサブネットが配置済み。検証・学習には便利だが、本番環境では **専用の VPC を自分で作る** のが望ましい。

4.2 サブネットとルートテーブル

4.2.1 パブリックサブネットとプライベートサブネット

技術的には両者に本質的な違いはない。ルートテーブルが Internet Gateway にルーティングしているかどうかで決まる。

種類	特徴
パブリックサブネット	Internet Gateway へのルートを持つ。パブリック IP を持てば外部から到達可能
プライベートサブネット	Internet Gateway へのルートを持たない。外部からは到達不可能
NAT 経由のプライベート	NAT Gateway 経由で外向き通信のみ可能（着信は不可）
隔離サブネット	外部との通信を一切持たない（DB 専用など）

4.2.2 ルートテーブルの役割

ルートテーブルは、各サブネットからのパケット転送先を決める。

パブリックサブネット用のルートテーブルの例

送信先	ターゲット	
10.0.0.0/16	local	# VPC 内部
0.0.0.0/0	igw-0abc...	# デフォルトルートを IGW に

プライベートサブネット（NAT経由）のルートテーブル

送信先	ターゲット	
10.0.0.0/16	local	
0.0.0.0/0	nat-0def...	# デフォルトルートを NAT Gateway に

1つのサブネットに関連付けられるルートテーブルは1つ。ルートテーブル自体は複数のサブネットで共有できる。

4.3 インターネット接続

4.3.1 Internet Gateway (IGW)

VPC をインターネットに接続するためのコンポーネント。VPC に1つだけアタッチする。IGW 自体は課金されない。

4.3.2 NAT Gateway

プライベートサブネットから **外向きだけ** インターネット接続するためのサービス。EC2 が OS パッケージ更新のためにインターネットに出る、といった用途で必須。

- ・ **時間課金 + データ転送課金** の両方がかかる
- ・ マルチ AZ 構成では、AZ ごとに NAT Gateway を置くのが推奨（耐障害性とデータ転送料削減のため）

- ・ 個人検証で立てっぱなしにすると **月数千円の固定費** になるので注意
- ・ 代替として NAT インスタンス（EC2 を NAT として使う）もあるが、運用負荷が上がるため推奨されない

4.3.3 外向き専用 Internet Gateway（Egress-Only IGW）

IPv6 の外向き専用 IGW。IPv6 には NAT の概念がないため、外向きだけ通す専用 GW が用意されている。

4.3.4 パブリック IP

- ・ **パブリック IPv4**：インスタンス停止で変わる。無課金（かつては）だが、**2024 年以降は使用有無にかかわらず時間課金** の扱いが拡大している
- ・ **Elastic IP**：固定のパブリック IP。インスタンスに紐づけている間は無料、未使用だと有料

4.4 セキュリティグループとネットワーク ACL

4.4.1 セキュリティグループ（SG）

ENI（Elastic Network Interface）単位のステートフル・ファイアウォール。EC2・RDS・ALB など、ネットワークインタフェースを持つリソースに適用される。

- ・ **ステートフル**：戻りトラフィックは自動で許可される
- ・ **許可ルールのみ**：明示的な拒否は書けない
- ・ **ENI あたり最大 5 つ** の SG をアタッチできる

典型的な SG ルールの書き方：

```
# Webサーバ用 SG
インバウンド：
  HTTP (80)   から 0.0.0.0/0
  HTTPS (443) から 0.0.0.0/0
  SSH (22)    から 203.0.113.0/24 # オフィス IP のみ

アウトバウンド：
  すべて     へ   0.0.0.0/0   (デフォルト)
```

4.4.2 SG 間参照

送信元に **別の SG** を指定できる。これを使うと「ALB の SG から来る通信だけ EC2 が受け付ける」のような層状の設計が IP 依存なしで組める。

```
# EC2 (Webサーバ) 側 SG
インバウンド：
  HTTP (80)   から sg-xxxxxx (ALB用SG)
```

4.4.3 ネットワーク ACL (NACL)

サブネット単位のステートレス・ファイアウォール。

- ・ステートレス：戻りトラフィックも明示的に許可する必要がある
- ・許可・拒否の両方が書ける
- ・ルールに番号を振り、小さい順に評価される

基本的にはセキュリティグループで制御し、NACLはデフォルトのまま運用する。特定IPをVPC全体でブロックしたい場合だけNACLを触る、という使い分けが現実的である。

4.5 VPC エンドポイント

VPC 内から AWS のマネージドサービス (S3、DynamoDB など) にアクセスする際、通常はインターネット経由となる。VPC エンドポイントを使うと、AWS 内部ネットワークを経由してサービスに到達できる。

4.5.1 ゲートウェイ型エンドポイント

- ・対象：S3 と DynamoDB のみ
- ・ルートテーブルにエントリが追加される方式
- ・無料

プライベートサブネットの EC2 から S3 にアクセスするために NAT Gateway を立てているなら、S3 ゲートウェイエンドポイントに置き換えるだけで NAT 通信量 (= 課金) が大幅に減る。個人利用でも設定しておくといよい。

4.5.2 インタフェース型エンドポイント

- ・対象：S3・DynamoDB 以外の多数のサービス (KMS、SQS、Secrets Manager 等)
- ・ENI がサブネット内に作られ、プライベート IP で到達できる
- ・時間課金 + データ処理課金

4.5.2.1 PrivateLink

インタフェース型エンドポイントの基盤技術。自社サービスを他アカウントからプライベートに公開することもできる。SaaS 事業者がよく使う。

4.6 VPC 間・オンプレ接続

4.6.1 VPC ピアリング

2つのVPCを直接接続する。シンプルだがハブ&スポーク構成には向かない(推移的ルーティング不可)。

4.6.2 Transit Gateway

複数VPC、オンプレ、VPN、Direct Connectを1つのハブにまとめて接続する。VPC数が増えたらこちら。

4.6.3 Site-to-Site VPN

オンプレ拠点と VPC を IPsec VPN で接続する。設定は比較的容易で、Direct Connect よりも低コスト。

4.6.4 Direct Connect

オンプレと AWS を専用線で接続する。高帯域・低レイテンシ・安定性が要件のケース向け。初期工事費や月額費用が発生する。

4.7 DNS と DHCP

4.7.1 VPC のデフォルト DNS

VPC には **Route 53 Resolver** が組み込まれており、`10.0.0.2` のような予約アドレスで DNS が解決される。EC2 は自動で `/etc/resolv.conf` にこれが設定される。

4.7.2 DNS ホスト名と DNS 解決

VPC の属性として以下 2 つを有効化しておくのが基本。

- **enableDnsSupport** : VPC 内で Amazon DNS を使う
- **enableDnsHostnames** : パブリック IP を持つインスタンスに `ec2-xx-xx-xx-xx.ap-northeast-1.compute.amazonaws.com` 形式のホスト名を付与

4.7.3 プライベートホストゾーン

Route 53 で VPC 内だけに見える独自ドメインを運用できる。`internal.example.com` のようなドメインを作って、社内リソースに名前を付けると可読性が上がる。

4.8 実践：小規模な Web 構成の VPC 設計

2 AZ の最小構成で、パブリック層 (ALB)・アプリ層 (EC2)・DB 層 (RDS) を分離する設計例。

VPC: `10.0.0.0/16` (ap-northeast-1)

AZ-a:

Public:	<code>10.0.1.0/24</code>	ALB
App:	<code>10.0.11.0/24</code>	EC2
DB:	<code>10.0.21.0/24</code>	RDS Master

AZ-c:

Public:	<code>10.0.2.0/24</code>	ALB
App:	<code>10.0.12.0/24</code>	EC2
DB:	<code>10.0.22.0/24</code>	RDS Standby

Internet Gateway: VPC にアタッチ

NAT Gateway: 各パブリックサブネットに1つ (AZ-a, AZ-c)

- Appサブネットの 0.0.0.0/0 は同じ AZ の NAT へ

セキュリティグループ:

sg-alb: 0.0.0.0/0 → 80/443

sg-app: sg-alb → 8080

sg-db: sg-app → 3306

VPCエンドポイント:

S3ゲートウェイ (App・DBのルートテーブル)

この構造を CloudFormation や Terraform でコード化してしまえば、検証環境を何度でも立て直せる。

4.9 VPC 作成時の注意点

- ・ CIDR は後から広げられるが、狭められない。最初に広め取る
- ・ 1つの AZ に寄せるとその AZ 障害で全滅する。最低 2 AZ を使う
- ・ NAT Gateway は固定費が発生する。検証では削除を忘れずに
- ・ サブネットの IP は、最初と最後の数個が AWS 予約 で使えない (/24 なら使えるのは 251 個)
- ・ Lambda を VPC に入れると ENI 起動分だけコールドスタートが伸びる 点に注意

VPC は一度作ったら壊しにくいので、最初から冗長性と拡張性を意識して設計する。

5 EC2（仮想マシン）

Amazon EC2（Elastic Compute Cloud）は、AWS で仮想マシンを起動・管理するサービスである。AWS の原点とも言える中心サービスであり、他のサービスを理解する上でも基本になる。

5.1 EC2 の基本概念

5.1.1 インスタンス

EC2 の **インスタンス** は、VPC 内のサブネットで起動する 1 台の仮想マシンである。起動時に以下を選択する。

- **AMI**：OS とソフトウェアの雛形
- **インスタンスタイプ**：CPU・メモリ・ネットワーク性能
- **キーペア**：SSH ログイン用の公開鍵／秘密鍵
- **ネットワーク**：VPC、サブネット、パブリック IP の有無
- **セキュリティグループ**：ファイアウォール設定
- **ストレージ（EBS）**：ルートボリュームと追加ディスク
- **IAM インスタンスプロファイル**：インスタンスから AWS API を呼ぶ権限
- **ユーザーデータ**：初回起動時に実行するスクリプト

5.1.2 料金体系

購入オプション	特徴
オンデマンド	秒／時間単位の従量課金。短期・予測困難な用途
Savings Plans	1～3 年のコミットで最大約 72% 割引。柔軟性が高い
Reserved Instance	従来型の予約購入。現在は Savings Plans 推奨
Spot	余剰キャパシティを最大 90% 割引で利用。中断あり
Dedicated Host	物理サーバを専有。BYOL ライセンス等で利用

個人検証では **オンデマンド** が基本。長期本番運用では **Savings Plans**、バッチ処理では **Spot** を組み合わせる。

5.2 AMI（Amazon Machine Image）

AMI は「OS + 初期ソフトウェア + ディスクイメージ」をまとめた雛形である。

5.2.1 主な AMI の出所

- **AWS 提供**：Amazon Linux 2023、Ubuntu、RHEL、Windows Server 等
- **AWS Marketplace**：サードパーティベンダのイメージ（WordPress、各種アプライアンス）
- **コミュニティ**：他ユーザーが公開した AMI
- **カスタム AMI**：自分で構築したインスタンスから作成

5.2.2 Amazon Linux 2023

AWS が提供する独自の Linux ディストリビューション。EC2 で使う場合の第一候補である。

- AWS SDK、CLI、SSM エージェントなどが同梱
- 最低 5 年のサポート
- `dnf` ベースのパッケージ管理（Fedora / RHEL 系）
- セキュリティパッチが AWS リポジトリから即座に配信される

5.2.3 カスタム AMI の作り方

ベースインスタンスに必要なソフトを入れて、そのイメージをスナップショット化する。

```
# 対象インスタンスから AMI を作成
aws ec2 create-image \
  --instance-id i-0abc123... \
  --name "my-app-v1.0" \
  --description "app baked on 2026-04-21"
```

作成した AMI から複数インスタンスを立ち上げれば、初期構成時間が大幅に短縮される。本番運用では Packer などで AMI をビルドパイプライン化するのが一般的。

5.3 インスタンスタイプ

EC2 のインスタンスタイプは `<ファミリー><世代><サイズ>` の命名規則を持つ。例： `m7i.large`、`c7g.xlarge`。

5.3.1 主要ファミリー

ファミリー	用途	例
T	バースト性能。軽量 Web、検証用	t3.micro, t4g.small
M	汎用。メモリと CPU がバランス	m7i.large
C	コンピュート最適化。CPU 重視	c7i.xlarge
R	メモリ最適化。キャッシュ、DB	r7i.large
X	極大メモリ。インメモリ DB	x2iedn.xlarge
I	ローカル NVMe SSD。高 IO DB	i4i.large
G/P	GPU。機械学習、グラフィック	g5.xlarge, p5.48xlarge

5.3.2 世代と CPU アーキテクチャ

世代番号は大きいほど新しく、基本的に同じ価格でも性能が高い。

- サフィックス `g`（例： `t4g`、`m7g`）は **Graviton**（ARM）プロセッサ。x86 よりコスパが良いことが多い
- サフィックス `i` は Intel、`a` は AMD
- サフィックス `n` はネットワーク強化、`d` はローカル NVMe 付き

アプリが ARM 対応なら、まず **Graviton** を試す価値がある。

5.3.3 t ファミリーのクレジット仕様

T インスタンスは **CPU クレジット** でバースト動作する。

- 通常はベースライン CPU 使用率（例：t3.micro は 10%）で稼働
- 余った分がクレジットとして蓄積
- クレジットを消費することで一時的に 100% まで跳ね上げられる
- クレジット枯渇時は **Standard モード**ではベースラインに戻る、**Unlimited モード**では追加課金で持続

CI や機械学習など CPU を継続的に使う用途では t ファミリーを避け、M/C ファミリーを使う。

5.4 ストレージ（EBS / インスタンスストア）

5.4.1 EBS（Elastic Block Store）

ネットワーク越しに接続される永続ブロックストレージ。インスタンスを止めてもデータは残る。

ボリュームタイプ	特徴
gp3	汎用 SSD。コスパが高い。通常はこれ
gp2	旧世代の汎用 SSD。新規では gp3 を使う
io2 / io2 Block Express	高 IOPS 要件の本番 DB 用
st1	スループット最適化 HDD。ログ・DWH
sc1	コールド HDD。低頻度アクセス

- スナップショットを S3 に保存（裏側）してバックアップできる
- スナップショットから別 AZ・別リージョンへボリュームを復元できる
- 暗号化は **常に有効化** を推奨（デフォルト暗号化をアカウント設定でオンにできる）

5.4.2 インスタンスストア

一部のインスタンスタイプに付属する **ローカル NVMe SSD**。

- 高速だが **インスタンスを停止・終了するとデータが消える**
- キャッシュ・一時ファイル・シャーディングされた DB など、消えてもよいデータに使う

5.4.3 ルートボリュームの扱い

デフォルトでは **インスタンス終了時にルート EBS も削除** される。データ保持したい場合は「終了時に削除」のチェックを外すか、別途データ用ボリュームを使う。

5.5 ネットワーク関連

5.5.1 キーペアと SSH

EC2 にログインするために、起動時に **キーペア**（公開鍵・秘密鍵のペア）を関連付ける。

```
# キーペアを作成して秘密鍵を取得
aws ec2 create-key-pair --key-name my-key \
  --query 'KeyMaterial' --output text > ~/.ssh/my-key.pem
chmod 400 ~/.ssh/my-key.pem

# Amazon Linux へ SSH
ssh -i ~/.ssh/my-key.pem ec2-user@<パブリックIP>
```

キーペア紛失時は、新しいインスタンスを立てて EBS を付け替えて復旧する、という手順になる。パスワード認証は基本使わない。

5.5.2 SSM Session Manager（推奨）

SSH を開かない 現代的な接続方式。Systems Manager 経由でインスタンスにコンソール接続する。

- ポート 22 を開放不要
- SSH 鍵の管理不要
- IAM で接続権限を制御
- 全コマンドが CloudTrail に記録される

前提：インスタンスに SSM エージェント（Amazon Linux 2023 は標準搭載）と、IAM ロール `AmazonSSMManagedInstanceCore` を付与する。

```
# CLIから接続
aws ssm start-session --target i-0abc123...
```

検証・本番ともに、**SSM Session Manager** を**第一選択** にすると運用が軽くなる。

5.5.3 Elastic IP

固定のパブリック IPv4 アドレス。インスタンスを再起動しても IP が変わらなくする。

- アタッチ中のインスタンスが稼働中なら無料
- 未使用（または停止中インスタンスに関連付け）は時間課金
- 使わないのに確保したまま放置しない

5.6 Auto Scaling とロードバランサ

5.6.1 ロードバランサ (ELB)

種類	用途
ALB (Application)	L7 / HTTP(S)。パス・ホスト・ヘッダでルーティング
NLB (Network)	L4 / TCP・UDP。超低レイテンシ・高スループット
GWL (Gateway)	セキュリティアプライアンスの挿入用
CLB (Classic)	旧世代。新規では使わない

Web アプリなら基本は ALB。 `/api/*` を別のターゲットグループに振り分ける、マイクロサービスのエッジなどでよく使われる。

5.6.2 ターゲットグループとヘルスチェック

ALB / NLB は **ターゲットグループ** を介して EC2 やコンテナにトラフィックを転送する。ヘルスチェック（例： `/health` に HTTP 200）でターゲットの生死を判定し、不健全なものを切り離す。

5.6.3 Auto Scaling グループ (ASG)

EC2 の台数を自動で増減させる。

- ・ **起動テンプレート** で AMI・インスタンスタイプ・ユーザーデータを定義
- ・ **最小／希望／最大** 台数を設定
- ・ **CPU 使用率** や **リクエスト数** などのメトリクスでスケーリング
- ・ ELB と連携して、新規インスタンスを自動的にターゲットグループに登録

例：平常時 2 台、CPU 70% 超で最大 10 台まで増える、というポリシーが典型的。

5.7 ユーザーデータと初期化

起動時に **ユーザーデータ** にシェルスクリプトを入れると、インスタンス起動直後に一度だけ実行される。

```
#!/bin/bash
dnf update -y
dnf install -y nginx
systemctl enable --now nginx
cat > /usr/share/nginx/html/index.html <<EOF
<h1>Hello from $(hostname)</h1>
EOF
```

繰り返し使う設定は **AMI に焼き込む**、ユーザーデータは **AMI 間の差分** を埋める用途に使うのが基本。

5.8 実習：ALB + EC2 の最小構成

1. 4章で作ったパブリックサブネット × 2 を使う
2. 起動テンプレートを作成（AMI、`t3.micro`、SG `sg-app`、ユーザーデータで `nginx` 起動）
3. ASG を最小 1・最大 2 で作成
4. ALB を作成、ターゲットグループに ASG を登録
5. ALB の DNS 名にアクセスして `nginx` ページが返れば成功
6. 確認後は ASG を 0 台に縮める か まとめて削除

実習後は Elastic IP、NAT Gateway、停止中の EBS を見回って、不要ならすぐ削除する。これが月末の請求事故を防ぐ最大のコツである。

5.9 トラブルシューティング

症状	確認ポイント
SSH できない	SG の 22 番開放、キーペア、パブリック IP、ルートテーブル
起動直後に停止する	インスタンスタイプの空き枯渇、IAM 権限不足、ユーザーデータ失敗
ALB でヘルスチェック失敗	SG (ALB → EC2)、ヘルスチェックのパスとポート、起動の遅延
起動が Pending のまま	AZ のキャパ不足。別 AZ を試す
AMI が世代不一致で失敗	AMI が x86 か ARM か、インスタンスタイプと合っているか
ディスク容量不足	EBS を拡張 → OS 側で <code>growpart</code> / <code>xfs_growfs</code>

6 S3（オブジェクトストレージ）

Amazon S3（Simple Storage Service）は、インターネット経由でアクセスできるオブジェクトストレージである。AWS で最も古いサービスの一つで、事実上無制限の耐久性・スケーラビリティを持ち、ログ保管から動画配信、データレイク、静的サイトホスティング、バックアップまで幅広く使われる。

6.1 S3 の基本概念

6.1.1 バケットとオブジェクト

- ・ **バケット**：オブジェクトを格納する最上位の入れ物。名前はグローバルに一意
- ・ **オブジェクト**：ファイル本体＋メタデータ＋キー（ファイルパスに相当する文字列）
- ・ **キー（Key）**：バケット内でオブジェクトを一意に識別する文字列（例：`logs/2026/04/21/app.log`）

S3 には「ディレクトリ」は存在しない。キーに `/` を含めることで、**あたかもディレクトリ構造があるように見える** だけである。

6.1.2 耐久性と可用性

- ・ **耐久性（Durability）**：99.999999999%（イレブンナイン）。年間で 1 万個に 1 個失う確率が約 0.0000001%
- ・ **可用性（Availability）**：Standard は年間 99.99%（設計上）
- ・ 内部では **複数 AZ に冗長コピー** されるため、AZ 障害を吸収できる

6.1.3 バケット名の制約

- ・ 3～63 文字、英小文字・数字・ハイフンのみ
- ・ **グローバルで一意**（他人が `company-backup` を使っていたら作れない）
- ・ 一度作ったら変更不可。削除して作り直す必要がある

命名規則は `<組織>-<環境>-<用途>-<リージョン>` のようにしておくとう衝突しづらい（例：`acme-prod-logs-apne1`）。

6.2 ストレージクラス

用途に合わせてストレージクラスを選ぶことで、大幅にコスト削減できる。

クラス	特徴
S3 Standard	デフォルト。頻繁アクセス用
S3 Intelligent-Tiering	アクセスパターン不明な場合。自動で階層移動
S3 Standard-IA (低頻度)	月 1 回程度の読み出し。最小 30 日課金
S3 One Zone-IA	1 AZ のみ。再生成可能な二次データ
S3 Glacier Instant Retrieval	年 1 回程度のアクセスで即時取得
S3 Glacier Flexible Retrieval	分～時間単位の取り出し時間を許容

クラス	特徴
S3 Glacier Deep Archive	12 時間程度の取り出し。最安。長期アーカイブ
S3 Express One Zone	超低レイテンシ。ML 訓練などの高 IO 用

個人検証では **Standard** で十分。大量ログやバックアップを貯めるなら **Glacier Deep Archive** にすると TB あたり月 \$1 程度まで落ちる。

6.2.1 ライフサイクルルール

オブジェクトの経過日数に応じて、自動的にストレージクラスを移動したり削除したりできる。

```
{
  "Rules": [{
    "Id": "archive-old-logs",
    "Status": "Enabled",
    "Filter": { "Prefix": "logs/" },
    "Transitions": [
      { "Days": 30, "StorageClass": "STANDARD_IA" },
      { "Days": 90, "StorageClass": "GLACIER" },
      { "Days": 365, "StorageClass": "DEEP_ARCHIVE" }
    ],
    "Expiration": { "Days": 2555 }
  }]
}
```

これは「logs/ 配下を 30 日で IA、90 日で Glacier、1 年で Deep Archive、7 年で削除」というルール。ログ系は必ずライフサイクルルールを入れる。

6.3 操作の基本

6.3.1 CLI によるアップロード／ダウンロード

```
# バケット一覧
aws s3 ls

# バケット作成
aws s3 mb s3://my-bucket-20260421 --region ap-northeast-1

# ファイルアップロード
aws s3 cp ./report.pdf s3://my-bucket-20260421/reports/

# ディレクトリ再帰アップロード
aws s3 sync ./dist s3://my-bucket-20260421/site/ --delete

# ダウンロード
aws s3 cp s3://my-bucket-20260421/reports/report.pdf ./
```

```
# 削除
aws s3 rm s3://my-bucket-20260421/reports/report.pdf

# バケット削除（空でない場合は --force）
aws s3 rb s3://my-bucket-20260421 --force
```

`aws s3api` は低レベル API、`aws s3` は高レベルのラップ。普段使いは `aws s3` が便利。

6.3.2 プリサインド URL

一時的にオブジェクトへのアクセスを許可する URL。アプリから直接 S3 にアップロード／ダウンロードさせる用途で多用する。

```
aws s3 presign s3://my-bucket-20260421/reports/report.pdf \
  --expires-in 3600
```

URL の有効期間内は、認証なしでそのオブジェクトだけにアクセスできる。ログインユーザーに限定公開するファイル配信などに便利。

6.4 アクセス制御

S3 のアクセス制御は歴史的経緯から **複数のレイヤ** が重なっている。これが初心者の混乱の原因になりがち。

6.4.1 階層

レイヤ	役割
ブロックパブリックアクセス	公開設定を全面禁止する強力なスイッチ（推奨 ON）
IAM ポリシー	IAM ユーザー／ロール側からバケットへの権限
バケットポリシー	バケット側に貼るリソースベースポリシー
オブジェクト ACL	旧方式。原則使わない（無効化が推奨）
S3 Object Ownership	バケット所有者に所有権を寄せる設定（推奨）

6.4.2 推奨デフォルト

新規バケットは以下の設定で作る。

- **Block all public access** : ON
- **S3 Object Ownership** : Bucket owner enforced（ACL 無効化）
- **バケット暗号化** : SSE-S3 または SSE-KMS を有効化（AWS 管理キーで十分）
- **バージョニング** : 必要に応じて有効化

特別な理由（静的サイト公開など）がなければ、まずはこの「**全部閉じる**」状態にして、必要な穴だけバケットポリシーで開ける。

6.4.3 バケットポリシーの例

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowReadFromVPCE",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::my-bucket/*",
      "Condition": {
        "StringEquals": {
          "aws:SourceVpce": "vpce-0abc123..."
        }
      }
    }
  ]
}
```

「特定の VPC エンドポイントから来るリクエストだけ GetObject を許可」という制御。Principal に `*` を書いても Condition で絞っていれば安全。

6.4.4 ブロックパブリックアクセスのバイパス

公開バケット（静的サイトなど）が必要なときだけ、明示的にブロックを解除する。

```
aws s3api put-public-access-block \
  --bucket my-public-site \
  --public-access-block-configuration \

  "BlockPublicAcls=false,IgnorePublicAcls=false,BlockPublicPolicy=false,RestrictPublicBuckets=false"
```

この操作自体が イベント通知されるように CloudTrail で監視 することが望ましい。

6.5 バージョニングとオブジェクトロック

6.5.1 バージョニング

バケットごとにオン／オフを切り替える。有効にすると、上書きや削除しても過去バージョンが残る。

- ・ 誤削除からの復旧： `DeleteMarker` を外せば元に戻せる
- ・ ランサムウェア対策：攻撃者が削除してもバージョンが残る
- ・ ストレージ料金が増える ため、ライフサイクルルールで古いバージョンを削除する

6.5.2 オブジェクトロック（WORM）

法規制対応のための Write-Once-Read-Many。一度書いたら指定期間削除・上書き不可にする。

- ・ **Governance** モード：特定権限を持つ者は解除可能
- ・ **Compliance** モード：ルートユーザーでも解除不可

監査ログや金融系のエビデンスを改ざんから守る用途で使う。

6.6 静的ウェブサイトホスティング

S3 バケットで静的サイト（HTML/CSS/JS）を配信できる。

1. バケットを作成、**ブロックパブリックアクセスを解除**
2. **プロパティ** → **静的ウェブサイトホスティング** を有効化
3. インデックスドキュメントを `index.html`、エラーを `error.html` に設定
4. バケットポリシーで `s3:GetObject` を `*` に許可
5. `http://<bucket>.s3-website-<region>.amazonaws.com` でアクセスできる

ただし S3 単体では HTTPS を提供しない。**本番では CloudFront + ACM** を前段に置いて HTTPS 化するのが標準構成。

```
[ユーザー] → CloudFront (HTTPS, キャッシュ)
               ↓ OAC (Origin Access Control)
               [S3 バケット (非公開)]
```

この構成なら S3 はブロックパブリックアクセスを維持したまま、CloudFront 経由だけ許可できる。

6.7 イベント通知

S3 へのオブジェクト作成・削除をトリガーに、Lambda / SQS / SNS / EventBridge を起動できる。

- ・ 画像アップロード → Lambda でサムネイル生成
- ・ ログ到着 → SQS 経由でワーカーへ
- ・ 不正なアップロードを EventBridge で検知・通知

これは「S3 をイベント起点にした非同期処理」の定番パターン。サーバーレス構成と相性がよい。

6.8 クロスリージョンレプリケーションと S3 Replication Time Control

耐災害性のために、別リージョンへ自動コピーする機能。

- ・ **CRR**：クロスリージョンレプリケーション
 - ・ **SRR**：同一リージョンレプリケーション（別バケットへ）
 - ・ バージョニング有効が前提
 - ・ **S3 RTC** 付きなら 99.99% のオブジェクトが 15 分以内にレプリケーションされる SLA を得られる
- DR（Disaster Recovery）要件があれば検討する。

6.9 コスト最適化のポイント

- ・ ライフサイクルで階層化：古いものを IA → Glacier → Deep Archive に流す
- ・ Intelligent-Tiering：アクセスパターンが読めないデータに一律適用
- ・ 不完全なマルチパートアップロードを消す：ライフサイクルで自動削除（放置すると永久に残る）
- ・ バージョニング有効バケットの古いバージョン削除：同じくライフサイクルで
- ・ データ転送課金に注意：別リージョンへの転送やインターネット配信は高い
- ・ S3 Storage Lens でバケット全体の使用状況を可視化

6.10 S3 のよくあるつまずき

症状	対処
403 AccessDenied	ブロックパブリックアクセス、バケットポリシー、IAM ポリシーを順に確認
バケットが作れない	グローバル一意の名前にする、DNS 準拠の名前にする
料金が想定外	Cost Explorer で「S3 > リクエスト課金」「データ転送」を分析
削除できない	バージョニング有効バケット内は、全バージョンと DeleteMarker を消す
アップロード遅い	マルチパート、Transfer Acceleration、並列数を調整

7 データベース

AWS は多様なデータベースサービスを提供している。リレーショナル、NoSQL、インメモリ、時系列、グラフなど、データモデルと負荷特性に合わせて選択する「**Purpose-Built Database**」という思想である。本章では代表的な 4 系統を扱う。

7.1 サービス選択の早見表

サービス	用途	モデル
RDS	汎用 RDBMS (MySQL、PostgreSQL、Oracle、SQL Server、MariaDB)	リレーショナル
Aurora	RDS 上位版。クラウドネイティブな MySQL / PostgreSQL 互換	リレーショナル
DynamoDB	サーバーレス KVS / ドキュメント。超スケーラブル	NoSQL (KVS)
ElastiCache	Redis / Valkey / Memcached のマネージド	インメモリ
DocumentDB	MongoDB 互換	ドキュメント
Neptune	グラフ DB	グラフ
Timestream	時系列データ	時系列
Redshift	データウェアハウス	列指向
Athena	S3 上のデータに SQL	サーバーレス SQL
OpenSearch Service	全文検索・ログ分析	検索エンジン

まずは **RDS / Aurora / DynamoDB / ElastiCache** の 4 つを押さえれば、ほとんどのユースケースに対応できる。

7.2 RDS (Relational Database Service)

RDS は定番 RDBMS をマネージドで提供する。バックアップ・パッチ適用・フェイルオーバーを AWS 側に寄せられる。

7.2.1 サポートエンジン

- MySQL、PostgreSQL、MariaDB (オープンソース系)
- Oracle、SQL Server (商用。ライセンス持ち込み / 含みを選べる)

新規アプリでは PostgreSQL か Aurora MySQL / Aurora PostgreSQL が選ばれることが多い。

7.2.2 インスタンスクラスとストレージ

- インスタンスクラス: `db.t3.micro`、`db.m7g.large` など EC2 と同じ命名規則
- ストレージタイプ: `gp3` (汎用 SSD、デフォルト)、`io1/io2` (高 IOPS)、`magnetic` (旧式)
- ストレージは後から拡張可能、縮小は不可

7.2.3 高可用性：Multi-AZ 配置

Multi-AZ を有効にすると、別 AZ にスタンバイレプリカが同期レプリケーションされる。

- ・ 障害時は自動で DNS 切り替え（通常 1～2 分）
- ・ 単一エンドポイント（`<name>.<id>.<region>.rds.amazonaws.com`）を使い続ける
- ・ スタンバイ側は読み取りには使えない（Aurora と違う点）

本番では **Multi-AZ 必須**、検証では無効にしてコストを抑える。

7.2.4 リードレプリカ

読み取りを分散するための非同期レプリカ。Multi-AZ と別物で、以下の特徴がある。

- ・ 最大 15 台（MySQL / PostgreSQL）
- ・ 別リージョンにも置ける（DR 用途）
- ・ レプリカを昇格させて独立 DB にすることも可能

7.2.5 バックアップと復元

- ・ **自動バックアップ**：デフォルト 7 日、最大 35 日。ポイントインタイム復元（PITR）が可能
- ・ **手動スナップショット**：任意のタイミングで取る。別リージョンにコピー可
- ・ **AWS Backup** で横断管理も可能

PITR は「3 日前の午前 10 時 15 分」のような粒度で復元できる、便利機能。

7.2.6 パラメータグループとオプショングループ

- ・ **パラメータグループ**：`max_connections` など DB 設定を管理
- ・ **オプショングループ**：Oracle / SQL Server 向けの追加機能を管理

本番環境では **デフォルトのパラメータグループを使わず、自前のものを作る** と後から変更しやすい。

7.3 Aurora

RDS と同じ感覚で使えるが、ストレージが **独自の分散ストレージ層** になっている。

7.3.1 Aurora の特徴

- ・ **MySQL 5.7/8.0 互換、PostgreSQL 互換** の 2 系統
- ・ **3 AZ × 2 コピー = 6 コピー** を AWS 側で自動管理
- ・ 1 つのクラスタ内に最大 15 台のリーダーを置ける
- ・ **最大 128 TiB** まで自動拡張
- ・ **Aurora Serverless v2** では負荷に応じて自動でスケール

7.3.2 Aurora Serverless v2

ACU（Aurora Capacity Unit）単位で自動スケールする。

- ・ 0 ACU（ゼロスケール）まで落とせる世代もあり、使わない時間は課金ゼロに近づく
- ・ 検証・低頻度利用・スパイクがあるワークロードに向く

- Multi-AZ 相当の可用性

7.3.3 Aurora Global Database

複数リージョンにまたがるレプリケーション。典型的には主リージョン+副リージョンで DR を構成する。物理ストレージレベルのレプリケーションで、通常 1 秒未満の RPO を実現する。

7.4 DynamoDB

フルマネージドの NoSQL KVS / ドキュメントストア。サーバーレスで、スキーマ設計さえ合えば極端にスケールする。

7.4.1 基本構造

- テーブル、アイテム（行に相当）、属性（列に相当）
- パーティションキー（必須）と ソートキー（任意）で主キーを構成
- セカンダリインデックス：GSI（グローバル）と LSI（ローカル）

7.4.2 料金モード

モード	特徴
オンデマンド	リクエスト単位の課金。スパイク・不定期アクセスに
プロビジョンド	秒あたりの容量（RCU/WCU）を予約。Auto Scaling 可

新規プロジェクトは **オンデマンド** で始めて、負荷が読めてきたらプロビジョンドに移すのが無難。

7.4.3 Single Table Design

DynamoDB では RDB 的な「テーブルごとに正規化」は不向き。**1 テーブルに複数エンティティを詰め込み、アクセスパターンごとにキー設計する** という思想が推奨される。学習コストは高めだが、これを受け入れるとスループットとコストが大きく変わる。

7.4.4 DynamoDB Streams

テーブルへの変更を 24 時間キャプチャする。Lambda をトリガーに、監査ログ出力、キャッシュ更新、他サービス連携などが組める。

7.4.5 Global Tables

複数リージョンのテーブルを双方向でレプリケーションする。リージョン障害時にもアプリが動き続ける「アクティブ／アクティブ」構成。

7.5 ElastiCache

インメモリデータストアのマネージド。Redis / Valkey / Memcached をサポート。

7.5.1 選び方

- Valkey / Redis：機能豊富。永続化、Pub/Sub、トランザクション、各種データ構造

- **Memcached**：シンプル KVS。マルチスレッドで純粋なキャッシュ用途

新規利用では **Valkey**（Redis フォーク、より積極的にメンテされている）か **Redis** を選ぶ。

7.5.2 典型的なユースケース

- **セッションストア**：ALB + EC2 の後ろに置いて、セッションを共有
- **DB 前面キャッシュ**：RDS / Aurora の読み取りをキャッシュ
- **レートリミット**：API Gateway 前段での制限カウント
- **ランキング／リアルタイム集計**：Redis の Sorted Set を活用

7.5.3 クラスタモード

- **クラスタモード無効**：1つのプライマリ+レプリカ。小～中規模に十分
- **クラスタモード有効**：シャーディングで水平スケール。超大規模向け

7.6 DB 選定のガイドライン

7.6.1 「まず RDBMS」で始める

アクセスパターンが読めない段階では、**Aurora PostgreSQL** のような柔軟な RDBMS で始め、ボトルネックが見えてから DynamoDB / ElastiCache を足すのが安全である。初手 DynamoDB は、アクセスパターンを読み違えるとスキーマ設計のやり直しになりやすい。

7.6.2 マネージドかセルフホストか

EC2 に自前で MySQL を立てるのは、ほぼ非推奨。バックアップ、フェイルオーバー、バージョンアップ、パッチ適用 すべて自前で面倒を見ることになり、マネージドサービスの料金差をすぐ上回るコストになる。個人検証でも Aurora Serverless v2 などフルマネージドを使うのがよい。

7.6.3 商用ライセンスの扱い

Oracle / SQL Server は **ライセンスインクルード** か **BYOL（持ち込み）** を選べる。既存ライセンス資産があれば BYOL、なければインクルードで試算する。

7.7 DB 周りのネットワーク

RDS / Aurora / ElastiCache は通常 **プライベートサブネット** に置く。

- DB サブネットグループ（2 つ以上の AZ を指定）を作って、その中に配置
- SG で **アプリサーバの SG からのみ接続可** にする
- **パブリックアクセスを有効化しない**（個人検証でも基本禁止）

社外から接続したい場合は、踏み台 EC2 や **SSM セッションマネージャ** のポートフォワーディングを使う。

```
# ローカルから RDS に SSM 経由でトンネリング
aws ssm start-session \
```

```
--target i-0bastion... \  
--document-name AWS-StartPortForwardingSessionToRemoteHost \  
--parameters '{"host":["mydb.xyz.ap-northeast-1.rds.amazonaws.com"],"portNumber":  
["5432"],"localPortNumber":["15432"]}'
```

7.8 バックアップとリカバリのベストプラクティス

- RPO / RTO を明確にする：どれくらいのデータ損失と復旧時間を許容するか
- ポイントインタイム復元 を有効化
- 定期的なリストア演習 をする：取っただけのバックアップは「使えるか」分からない
- クロスリージョンスナップショットコピー でリージョン災害に備える
- IAM で削除・暗号化解除を絞る：誰でも `DeleteDBInstance` できる状態にしない

7.9 コスト最適化のポイント

- アイドル DB を止める または削除する。開発用はスケジュールで夜間停止
- Aurora Serverless v2 で低負荷期間を 0 ACU に近づける
- IOPS を無闇に上げない。gp3 の場合、スループット・IOPS も課金対象
- リードレプリカの数を見直す：余計なレプリカは費用がかさむ
- DynamoDB のプロビジョンドキャパシティを Auto Scaling で最適化

8 コンテナとサーバーレス

EC2 で仮想マシンを立てる方法に加えて、AWS では **コンテナ** や **サーバーレス** でアプリを動かす選択肢がある。運用負荷を AWS に寄せるほど、開発者はアプリケーションロジックに集中できる。本章ではこの領域の主要サービスを俯瞰する。

8.1 コンピュート選択のスペクトラム

レイヤ	責任範囲（利用者側）	例
仮想マシン	OS・ミドルウェア・アプリ	EC2
コンテナ（IaaS）	コンテナ・タスク定義	ECS on EC2、EKS on EC2
コンテナ（サーバーレス）	コンテナだけ	ECS on Fargate、EKS on Fargate
サーバーレス関数	関数コードだけ	Lambda
フルマネージド PaaS	アプリコードだけ	App Runner、Elastic Beanstalk、Amplify

下へ行くほど運用負荷が下がり、抽象度が上がる。

8.2 Lambda（サーバーレス関数）

AWS Lambda は、関数単位でコードを実行するサーバーレス基盤である。インフラを意識せず、イベント駆動で動くコードを書ける。

8.2.1 実行モデル

- ・コードをランタイム（Python、Node.js、Java、Go、Ruby、.NET、Rust）で動かす
- ・コンテナイメージ（OCI）としてデプロイも可能（最大 10 GB）
- ・イベントをトリガーに起動：API Gateway、S3、SQS、EventBridge、DynamoDB Streams など
- ・同時実行数（Concurrency）でスケール。デフォルト上限はアカウント・リージョンで 1000

8.2.2 料金モデル

- ・リクエスト数 × 実行時間 × メモリサイズの従量課金
- ・メモリを上げると CPU も連動して増える（時間が短縮されれば相殺される場合も）
- ・月 100 万リクエスト、40 万 GB 秒までは **Free Tier**（常時無料）

8.2.3 コールドスタート

しばらく呼ばれていない Lambda は、次の呼び出し時に **コンテナ起動＋ランタイム初期化** が走る。これが **コールドスタート**。

- ・数十 ms ～ 数秒（言語・パッケージサイズに依存）
- ・解消策：Provisioned Concurrency（事前に温めておく）、SnapStart（Java で起動時間短縮）

- VPC 内 Lambda は ENI 初期化でさらに長くなる

8.2.4 シンプルな Lambda の例 (Python)

```
import json

def lambda_handler(event, context):
    name = event.get("queryStringParameters", {}).get("name", "world")
    return {
        "statusCode": 200,
        "body": json.dumps({"message": f"hello, {name}!"})
    }
```

デプロイは以下の通り。

```
zip -r func.zip lambda_function.py
aws lambda create-function \
  --function-name hello \
  --runtime python3.12 \
  --role arn:aws:iam::123456789012:role/lambda-basic \
  --handler lambda_function.lambda_handler \
  --zip-file fileb://func.zip
```

8.2.5 Lambda のユースケース

- API バックエンド：API Gateway + Lambda
- ファイル処理：S3 アップロード → Lambda でサムネイル生成・ウイルススキャン
- ストリーム処理：DynamoDB Streams、Kinesis
- スケジュールジョブ：EventBridge Scheduler で cron 相当
- ChatOps / Slack Bot：WebHook 受信処理
- IoT バックエンド：IoT Core からの軽い処理

8.2.6 制限

- 実行時間 15 分上限：長時間処理には向かない（Step Functions や Fargate と組み合わせる）
- メモリ上限 10 GB、一時ディスク 10 GB
- 同時実行数の制限：急激なスケールでスロットリングに注意

8.3 API Gateway

HTTP / REST / WebSocket の API をマネージドで提供する。Lambda、HTTP バックエンド、VPC Link 経由で NLB / ALB にも接続できる。

8.3.1 種類

- HTTP API：軽量・低レイテンシ・安価。新規は基本これ
- REST API：機能豊富（使用量プラン、API キー、WAF 連携、リクエスト検証）

- WebSocket API：双方向通信

8.3.2 Lambda Proxy 連携

API Gateway HTTP API の典型パターン。リクエストを Lambda にそのまま渡し、Lambda のレスポンスを HTTP レスポンスに変換する。

8.3.3 認証／認可

- Cognito オートライザー：ユーザー認証 (JWT)
- Lambda オートライザー：独自ロジックで認可
- IAM 認可：SigV4 署名付きリクエスト

8.4 ECS / EKS / Fargate

8.4.1 ECS (Elastic Container Service)

AWS 独自のコンテナオーケストレーション。**タスク定義** という概念で Docker コンテナの起動仕様を記述する。

- クラスター > サービス > タスク という階層
- タスクは EC2 上 または Fargate 上で動く
- ELB との統合、Auto Scaling、Blue/Green デプロイ対応

EKS より **シンプル・学習コストが低い**。AWS だけで完結する運用なら ECS の方が扱いやすいことが多い。

8.4.2 EKS (Elastic Kubernetes Service)

Kubernetes (k8s) のコントロールプレーンをマネージドで提供する。

- **Kubernetes のエコシステムをそのまま使える**
- オンプレや他クラウドとの移植性が高い
- 運用ノウハウが必要。IAM・ENI・CoreDNS・Ingress で AWS 固有の癖あり
- **Karpenter** を使うとノードスケールが賢くなる

「Kubernetes の知識があり移植性重視」なら EKS、「シンプルに回したい」なら ECS。

8.4.3 Fargate

コンテナを **サーバーレス** で動かす仕組み。EC2 ノードを自前で管理せず、必要な vCPU・メモリだけ指定してタスクを起動する。

- EC2 ノードの OS パッチやスケールから解放される
- **タスクの vCPU × 時間** で課金
- ネットワーク的には VPC の ENI が各タスクに付く

小～中規模で運用負荷を下げたい場合は **ECS on Fargate**、k8s 要件があるなら **EKS on Fargate** が第一候補。

8.4.4 ECR (Elastic Container Registry)

OCI 互換のコンテナレジストリ。Docker Hub の AWS 内版。

- パブリックレジストリ（誰でも pull 可）と プライベートレジストリ
- IAM でアクセス制御
- 脆弱性スキャン、イメージ署名
- ECS / EKS / Lambda（コンテナイメージ方式）から引かれる

```
# ECR にログイン
aws ecr get-login-password --region ap-northeast-1 \
| docker login --username AWS --password-stdin \
  123456789012.dkr.ecr.ap-northeast-1.amazonaws.com

# タグ付けして push
docker tag myapp:latest \
  123456789012.dkr.ecr.ap-northeast-1.amazonaws.com/myapp:latest
docker push \
  123456789012.dkr.ecr.ap-northeast-1.amazonaws.com/myapp:latest
```

8.4.5 ECS タスク定義の例

```
{
  "family": "web",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "512",
  "memory": "1024",
  "executionRoleArn": "arn:aws:iam::123456789012:role/ecsTaskExecutionRole",
  "containerDefinitions": [{
    "name": "nginx",
    "image": "123456789012.dkr.ecr.ap-northeast-1.amazonaws.com/myapp:latest",
    "portMappings": [{ "containerPort": 80 }],
    "logConfiguration": {
      "logDriver": "awslogs",
      "options": {
        "awslogs-group": "/ecs/web",
        "awslogs-region": "ap-northeast-1",
        "awslogs-stream-prefix": "web"
      }
    }
  ]
}
```

8.5 App Runner / Elastic Beanstalk / Amplify

より高レベルの PaaS 的サービス群。

8.5.1 App Runner

コンテナまたは GitHub リポジトリから URL 付きの Web アプリを直接起動できる。VPC や ALB の設定も自動でやってくれる。

- シンプルな Web アプリ（API、SSR フロントエンド等）に最適
- 料金は vCPU × 時間 + リクエスト数
- Fargate より抽象度が高い

8.5.2 Elastic Beanstalk

古くからある PaaS。アプリをアップロードすれば EC2 / ALB / RDS を含めて環境を自動構築。

- 機能的には枯れているが、新規採用は減少傾向
- 裏のリソースにアクセスできるのが特徴（Lock-in が少ない）

8.5.3 Amplify

フロントエンド・モバイル向けのフルスタックホスティング。

- Git 連携で自動ビルド・デプロイ
- 認証（Cognito）、API（GraphQL / REST）、ストレージ（S3）、ホスティング（CloudFront）を統合
- React / Vue / Next.js などの SSR / SPA をそのまま乗せられる

Next.js や Nuxt.js の個人プロジェクトをデプロイするなら Amplify または App Runner が楽。

8.6 イベント駆動・非同期アーキテクチャ

サーバーレス／コンテナと組み合わせて使われる重要サービス。

サービス	役割
SQS	メッセージキュー。Standard / FIFO の 2 種
SNS	Pub/Sub。トピックに発行すると複数宛に配信（メール・SMS・HTTP・SQS・Lambda）
EventBridge	イベントバス。AWS サービス + SaaS を統合
Step Functions	ステートマシン。Lambda / サービスを組み合わせたワークフロー
Kinesis	ストリーミング。Data Streams / Firehose / Analytics
MSK	マネージド Kafka

8.6.1 SQS + Lambda の基本パターン

1. Producer（Web / Lambda）が SQS にメッセージを送る
2. Lambda が SQS をポーリングして 1 件ずつ処理
3. 失敗時は DLQ（Dead Letter Queue）に退避

このパターンは、ピークをキューで吸収し、バックエンドを非同期化する定番。バーストトラフィックを受ける Web アプリで多用する。

8.6.2 Step Functions

Lambda や AWS サービス呼び出しを **ステートマシン** で繋ぐ。長時間処理、分岐、リトライ、並列実行を JSON で宣言できる。

```
[Start] → [画像解析Lambda] → (成功) → [サムネイル生成] → (並列) → [通知]
                      ↳ (失敗) → [エラーログ] → [End]
```

Lambda 単体の 15 分上限を超える処理や、複雑なフローの可視化に有用。

8.7 サーバーレス／コンテナの選定指針

- 単純な HTTP エンドポイント：API Gateway + Lambda、または App Runner
- バッチ処理・非同期ジョブ：SQS + Lambda、または ECS Fargate タスク
- 長時間処理：Step Functions または ECS Fargate タスク
- 既存 Docker アプリを動かす：ECS on Fargate
- Kubernetes 資産を持っている：EKS
- フロントエンド中心：Amplify / CloudFront + S3
- 自前でスケーリング込みで運用できる：EC2 + Auto Scaling

「まずは Lambda → 限界が見えたら ECS / Fargate → さらに複雑なら EKS」という段階的なステップアップが一つの筋道である。

8.8 参考アーキテクチャ：サーバーレス Web アプリ

```
[ユーザー]
  ↓ HTTPS
[CloudFront] — [S3 (静的フロント)]
  ↓ /api/*
[API Gateway]
  ↓
[Lambda]
  ↓
[DynamoDB]    [Cognito (認証)]
```

このスタックは **インフラを 1 台も立てずに** Web アプリを動かせる。コストもアクセスが少ないうちは Free Tier の範囲内で収まることが多く、学習・個人プロジェクトの定番になっている。

9 運用と監視

AWS で動くシステムを安定運用するには、**メトリクス・ログ・監査・設定・構成管理** の各観点で仕組みを整える必要がある。本章では CloudWatch、CloudTrail、Config、Systems Manager を中心に解説する。

9.1 CloudWatch

CloudWatch は AWS の監視基盤である。メトリクス、ログ、アラーム、イベントをまとめて扱う。

9.1.1 CloudWatch Metrics

- AWS サービスから **標準メトリクス** が自動的に流れてくる（EC2 の CPU、ELB のリクエスト数など）
- **カスタムメトリクス** をアプリから `PutMetricData` で送れる
- 標準は 1 分間隔、詳細モニタリングで 1 分、カスタムで最短 1 秒粒度
- **15 か月間保持**（古いほど粗い粒度に丸まる）

9.1.2 アラーム

メトリクスに対してしきい値を設定し、違反時にアクションを発動する。

- **アクション**：SNS 通知（メール、Slack 連携）、Auto Scaling、EC2 停止など
- **複合アラーム**：複数アラームの AND/OR 合成
- **異常検出**：機械学習による動的しきい値

```
# CPU 80% 超で通知するアラームの作成例
aws cloudwatch put-metric-alarm \
  --alarm-name web-cpu-high \
  --metric-name CPUUtilization \
  --namespace AWS/EC2 \
  --statistic Average \
  --period 300 \
  --threshold 80 \
  --comparison-operator GreaterThanThreshold \
  --evaluation-periods 2 \
  --dimensions Name=InstanceId,Value=i-0abc123 \
  --alarm-actions arn:aws:sns:ap-northeast-1:123456789012:ops-alerts
```

9.1.3 CloudWatch Logs

サービスやアプリのログを集約する。

- **ロググループ** と **ログストリーム** の 2 階層
- **保持期間** を明示的に設定しないと無期限（料金が際限なく伸びる）
- **収集源**：Lambda（自動）、ECS / EKS のタスク、EC2 の CloudWatch エージェント、VPC Flow Logs 等

9.1.4 Logs Insights

CloudWatch Logs への独自クエリ言語。

```
fields @timestamp, @message
| filter @message like /ERROR/
| sort @timestamp desc
| limit 100
```

SQL 的な書き味で、**急な障害解析に強い**。Athena + S3 に流し込む構成にしておくと、長期保管と分析を両立できる。

9.1.5 ダッシュボード

メトリクスとログの可視化をカスタマイズできる。1 アカウント 3 ダッシュボードまで無料。

9.1.6 CloudWatch Alarms の通知をメール以外に

- SNS → **Chatbot** 経由で Slack / Teams
- SNS → Lambda でカスタムフォーマット
- EventBridge → 任意の AWS アクションに分岐

9.2 CloudTrail

CloudTrail は「**誰が・いつ・何を**」の**監査ログ**を記録するサービスである。AWS アカウント作成時点でデフォルトで有効（直近 90 日分）。

9.2.1 イベント種別

- 管理イベント：API 操作（`CreateBucket`、`TerminateInstances` など）
- データイベント：S3 の `GetObject` / `PutObject`、Lambda の `Invoke` など高頻度イベント
- インサイトイベント：異常な API 呼び出しパターンの検知

9.2.2 証跡 (Trail)

長期保存や複数リージョン集約には**証跡**を作成する。

- S3 に保存（推奨はログアーカイブ専用アカウント）
- **ログファイル検証**を有効にして改ざん検知
- CloudWatch Logs へ流して Logs Insights で検索
- EventBridge と組み合わせて重要操作を即時通知

組織全体の監査ログは **Organizations 証跡**で一括管理できる。

9.2.3 主要ログフィールド

- `eventName`、`eventTime`、`sourceIPAddress`
- `userIdentity`（IAM ユーザーかロールかルートか、引き受けた経路）
- `userAgent`（AWS CLI、Console、SDK）
- `requestParameters` / `responseElements`

「ルートユーザーからの API が実行されたら即通知」のような監査アラートは、CloudTrail + EventBridge でよく組まれる。

9.3 AWS Config

AWS Config は、リソースの構成変更を時系列で記録し、ルール違反を検知する サービスである。

- 全リソースの **設定履歴** を S3 / DynamoDB に保存
- **マネージドルール**（例：「S3 バケットが公開されていないこと」「EBS が暗号化されていること」）
- **カスタムルール**（Lambda / Guard DSL で書く）
- **コンフォーマンスパック**：ルールの束（CIS ベンチマーク、PCI DSS など）

9.3.1 典型的な利用シーン

- セキュリティ基準違反の検知（公開 S3、未暗号化 EBS、MFA なしユーザー）
- 構成変更の時系列追跡（「このセキュリティグループはいつ誰が変えた？」）
- コンプライアンスレポートの自動生成

コストは **記録対象リソース数 × 変更回数** で増えるため、マルチアカウント環境では **Config アグリゲータ** で全体可視化と、必要なルールだけ有効化するバランスが重要。

9.4 Systems Manager (SSM)

AWS Systems Manager は、EC2 やオンプレサーバの **構成管理・運用自動化** を統合するサービス群である。非常に広いので主要コンポーネントを押さえる。

9.4.1 主要機能

コンポーネント	用途
Fleet Manager	インスタンスの一覧管理・インベントリ取得
Session Manager	ブラウザ／CLI からインスタンスへ安全に接続
Run Command	複数台にコマンドを一斉実行
Patch Manager	OS パッチの自動適用
Parameter Store	設定値・秘密情報の保存（階層型）
State Manager	定期的に構成をあるべき姿に収束
Automation	多段の運用タスクを Runbook 化
OpsCenter	運用インシデントの一元管理
Change Manager	変更申請・承認ワークフロー

9.4.2 Session Manager のメリット

前章の EC2 でも触れたが、**SSH 22 番を閉じたまま** 接続できる現代的な運用の起点になる。

- SSH 鍵管理が不要
- 22 番ポートをセキュリティグループで閉じられる
- すべての操作が CloudTrail に残る

- ・ポートフォワーディングで RDS やプライベート LB への接続も可能

9.4.3 Parameter Store

アプリケーションの設定値や認証情報を階層パスで保存する。

```
# パラメータの登録（暗号化）
aws ssm put-parameter \
  --name "/app/prod/db_password" \
  --value "supersecret" \
  --type SecureString

# 取得
aws ssm get-parameter \
  --name "/app/prod/db_password" \
  --with-decryption \
  --query Parameter.Value --output text
```

シンプルな設定値なら Parameter Store、より高機能なローテーションが必要ななら **Secrets Manager**（次章で扱う）を使い分ける。

9.4.4 Patch Manager

EC2 の OS パッチを定期適用する仕組み。

- ・パッチベースライン（どのパッチを当てるか）
- ・メンテナンスウィンドウ（いつ当てるか）
- ・パッチグループ（どのインスタンスに当てるか）

数台なら手動で十分だが、100 台規模になると必須。

9.5 監視・運用の全体像

AWS 上の典型的な構成では、以下が組み合わさる。

```
[アプリ] → CloudWatch Logs（アプリログ）
          ↓
          メトリクス → CloudWatch Alarms → SNS → Slack/メール
          ↓
          CloudTrail（API監査）→ S3 / CloudWatch Logs
          ↓
          Config（構成変更）→ S3 / ルール違反検知
```

これらをまとめて俯瞰する上位のサービスも存在する。

- ・ **CloudWatch Application Insights**：自動でアプリの健全性を検出
- ・ **X-Ray**：分散トレーシング
- ・ **Application Signals**：CloudWatch に統合された APM
- ・ **Amazon Managed Grafana / Prometheus**：OSS エコシステムをマネージドで
- ・ **Security Hub**：セキュリティ観点の検知を集約（次章）

- ・ CloudWatch Synthetics：外形監視（canary によるエンドツーエンド確認）

9.6 タグ戦略

運用を楽にする最大の投資が **タグ戦略** である。

- ・ **Name**：人間が識別するための名前
- ・ **Environment**： `prod` / `staging` / `dev`
- ・ **Owner**：運用責任者／チーム
- ・ **CostCenter / Project**：費用按分用
- ・ **ManagedBy**： `terraform` / `cdk` / `manual`

タグはリソース作成時に付けるのが鉄則。後から入れようとする数百リソースを見直すことになる。Organizations のタグポリシー で必須タグを強制できる。

9.7 アラート疲れを防ぐ

監視を入れて数週間経つと、アラートが日常の雑音になって誰も見なくなる 現象が起きる。

- ・ 本当に起こすべきもの だけをアラート化する（夜間に叩き起こされる価値があるか？）
- ・ シビアリティを分ける：警告は Slack、緊急は PagerDuty
- ・ 抑制・まとめ：同じアラートが 1 時間で 100 件来たら 1 件に集約
- ・ アクションを明文化 する：ランブックを Runbook として Automation に登録

9.8 運用のベストプラクティス

- ・ 最初から CloudTrail / Config / GuardDuty（次章）を有効化
- ・ ログ保持期間を必ず設定 する（特に CloudWatch Logs）
- ・ Systems Manager Session Manager を第一の接続手段 にする
- ・ 主要リソースに監視ダッシュボードを 1 枚作る（最初の障害対応が段違いに楽になる）
- ・ タグ付けルールを決めて自動化する
- ・ 月次で Trusted Advisor / Cost Explorer を見て棚卸しする

10 セキュリティとコスト管理

AWS は「使ってみる」ハードルは低いですが、**放置すると事故になる** 代表例がセキュリティとコストである。本章では、運用を続けるうえで欠かせない両領域のサービスと、運用プラクティスをまとめる。

10.1 セキュリティの全体像

AWS のセキュリティは層状に構成される。

【アイデンティティ層】	IAM / Identity Center / MFA / Access Analyzer
【データ層】	KMS / Secrets Manager / S3 暗号化 / EBS 暗号化
【ネットワーク層】	SG / NACL / WAF / Shield / PrivateLink
【検知・監査層】	CloudTrail / Config / GuardDuty / Security Hub
【対応層】	Systems Manager Incident Manager / EventBridge

個人検証ではここまで全部整える必要はないが、**本番運用では全レイヤに当たり前の対策を入れる**のが前提である。

10.2 KMS (Key Management Service)

KMS は、暗号鍵を管理するマネージドサービス。S3、EBS、RDS、Secrets Manager など多数のサービスが KMS を使って暗号化する。

10.2.1 キーの種類

- ・ **AWS マネージドキー**：サービスごとに自動生成される（例：aws/s3）。追加料金なし
- ・ **カスタマーマネージドキー (CMK)**：利用者が作成・管理するキー。ローテーション、キーポリシー、IAM ポリシーを細かく制御可能
- ・ **AWS 所有キー**：AWS 内部で完全に隠蔽されているキー

検証やシンプルな用途では AWS マネージドキーで十分。**監査要件や独立した権限管理が必要な場合は CMK を使う。**

10.2.2 キーポリシーと IAM

CMK は **キーポリシー**（KMS 鍵自身に付与するリソースベースポリシー）と IAM ポリシーの両方で制御される。鍵作成直後は、キーポリシーに明示的な許可がないと誰も使えなくなる点に注意。

10.2.3 エンベロープ暗号化

KMS は **データキー** を生成し、それで実データを暗号化する方式（エンベロープ暗号化）を推奨する。KMS 本体に大きなデータを投げず、データキーだけを守る仕組み。

10.3 Secrets Manager と Parameter Store

10.3.1 使い分け

項目	Parameter Store	Secrets Manager
基本料金	標準は無料	1 秘密あたり月 \$0.40
自動ローテーション	なし	あり (Lambda 連携)
クロスアカウント共有	限定	ネイティブ対応
レプリケーション	なし	複数リージョンへ
用途	設定値・軽い秘密	DB パスワード等、ローテーションが必要な秘密

個人検証では Parameter Store (SecureString) で十分。本番の DB パスワードなど、**ローテーション運用が必要な秘密情報** は Secrets Manager を使う。

10.3.2 DB パスワードのローテーション

Secrets Manager は RDS / Aurora / Redshift / DocumentDB と統合されていて、**ワンクリックで定期的なパスワードローテーション**を設定できる。裏では Lambda が古いパスワードで新しいパスワードを設定し直す。

10.4 WAF と Shield

10.4.1 AWS WAF

Web アプリケーションファイアウォール。CloudFront、ALB、API Gateway、App Runner の前段に貼れる。

- ・ **マネージドルール**：AWS・Marketplace ベンダが提供する既製ルールセット
- ・ **レート制限**：特定 IP からの過剰リクエストをブロック
- ・ **Bot Control**：自動化された不正アクセスの検出
- ・ **Geo ブロック**：国単位でアクセス制御

10.4.2 Shield / Shield Advanced

- ・ **Shield Standard**：AWS ユーザー全員が無料で利用。L3/L4 の基本 DDoS 対策
- ・ **Shield Advanced**：有償。L7 対策の強化、DDoS Response Team (DRT)、急増課金のクレジット保護

個人利用では Standard で十分。大規模・公開サービスで攻撃リスクが高い場合のみ Advanced を検討。

10.5 GuardDuty / Inspector / Macie / Security Hub

10.5.1 GuardDuty

脅威検知 サービス。CloudTrail、VPC Flow Logs、DNS ログを ML で分析し、不審な動きを検知する。

- Crypto マイニング痕跡、通信先の悪性 IP、IAM キー漏洩の兆候など
- アカウント単位でワンクリック有効化
- 個人検証なら最初の 30 日間は無料。その後も **月数ドル程度**

アカウント作成直後に有効化することを強く推奨。

10.5.2 Inspector

脆弱性スキャナ。EC2、ECR、Lambda を対象にパッケージ脆弱性（CVE）を継続検出する。

- EC2 は SSM エージェント経由
- ECR は push 時と定期
- 重大な脆弱性を EventBridge 経由で通知

10.5.3 Macie

S3 の **機密データ発見** を行う。個人情報（PII）やクレジットカード番号などのパターンを機械学習で検出。

10.5.4 Security Hub

上記のセキュリティ検知を **集約・統合** するハブ。CIS や PCI DSS などのセキュリティ基準に対するスコアも出す。

マルチアカウント環境では、**監査専用アカウントに Security Hub を集約** して組織全体を見るのが一般的。

10.6 IAM Access Analyzer

IAM のリソース共有・外部公開を検出するサービス。

- 外部公開されている S3 バケット、Lambda、ロールを洗い出す
- 想定外のクロスアカウントアクセスを警告
- CloudFormation にポリシー検証機能を組み込める

有効化コストは無料に近く、**アカウント作成直後に ON** にしておくと誤公開事故に早く気づける。

10.7 AWS Organizations と SCP

複数アカウントを運用する場合は **AWS Organizations** が中心になる。

10.7.1 管理構造

```

[管理アカウント (payer)]
├── OU: Security
│   ├── 監査ログ用アカウント
│   └── セキュリティツール用アカウント
├── OU: Workloads
│   ├── OU: Prod
│   │   └── prod-web アカウント
│   └── OU: Dev
│       └── dev-sandbox アカウント

```

- 各アカウントは **リソース・権限の隔壁** として機能
- **一括請求** (Consolidated Billing)：親アカウントに支払いを集約
- **Organizations 証跡** で CloudTrail を一括管理

10.7.2 SCP (サービスコントロールポリシー)

OU / アカウント単位で、そもそも使えるサービスや API を制限 する。IAM ポリシーより上位の「絶対の壁」として機能する。

```

{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Deny",
    "Action": "*",
    "Resource": "*",
    "Condition": {
      "StringNotEquals": {
        "aws:RequestedRegion": ["ap-northeast-1", "ap-northeast-3"]
      }
    }
  }]
}

```

このような SCP を付けておけば、**東京・大阪以外のリージョンで誤ってリソースが作られる** 事故を防げる。放置されたリソースがアラートに引っかからず料金が膨らむ、という典型事故の対策になる。

10.7.3 Control Tower

新規に組織を作るなら **AWS Control Tower** を使うと、Organizations / IAM Identity Center / ログアーカイブ構成 / ベースラインの SCP が一括で展開される。

10.8 コスト管理サービス

10.8.1 AWS Budgets

2 章でも触れた **予算アラート**。個人利用で最初に設定するサービス。

- **コスト予算**：月次・四半期・年次の金額上限
- **使用量予算**：EC2 時間、データ転送量など
- **Reservation 予算**：RI や Savings Plans の使用率管理
- アラートはメール・Slack（SNS 経由）・Chatbot

必ず「予測」ベースのアラートも入れる。実測が閾値に達してから通知では手遅れになる。

10.8.2 Cost Explorer

過去 13 か月～（有償で長期）のコスト・使用量を可視化・分析する。

- サービス別・タグ別・リージョン別で集計
- **Savings Plans 推奨** を自動算出
- カスタムレポートの保存、CSV エクスポート

月初に前月のコストを振り返る習慣をつけるだけで浪費を大きく減らせる。

10.8.3 Cost and Usage Report (CUR)

CUR 2.0（または CUR）は、もっと詳細な使用量・料金データを S3 に出すレポート。Athena / QuickSight と組み合わせて経営層向けのダッシュボードを作る用途が多い。

10.8.4 Savings Plans と Reserved Instance

- **Compute Savings Plans**：EC2 / Fargate / Lambda 横断。最大 66%
- **EC2 Instance Savings Plans**：特定ファミリーに縛る代わりに最大 72%
- **Reserved Instance**：旧来の予約。新規は Savings Plans が基本

安定的に稼働している計算リソースがあるなら、1 年コミットだけでもかなり下がる。

10.8.5 Spot Instance

Auto Scaling グループでオンデマンドと Spot を混ぜる、ECS で Fargate Spot を使う、バッチを AWS Batch で Spot に寄せる、といった使い方で大幅にコストが落ちる。中断耐性のあるワークロードから適用する。

10.8.6 コスト最適化のチェックリスト

- ☐ Budgets を設定した（実測＋予測）
- ☐ Cost Explorer を月 1 回見ている
- ☐ 停止した EC2 に紐づく EBS を残していないか
- ☐ 未使用の Elastic IP を解放したか
- ☐ NAT Gateway を検証用に立てっぱなしにしていないか
- ☐ CloudWatch Logs の保持期間を設定したか

- [] S3 バケットに Lifecycle ルールを設定したか
- [] Savings Plans の推奨を確認したか
- [] 全リージョンの「幽霊リソース」を点検したか

10.9 セキュリティ+コストで最低限やること

個人・小規模でも、最初の 30 分で 以下は終わらせる。

1. ルートユーザーに MFA、アクセスキーなし
 2. 作業用 IAM ユーザー or Identity Center を用意
 3. Budgets で予算アラート
 4. Free Tier 使用量アラート
 5. CloudTrail を有効化、S3 に長期保管
 6. GuardDuty を有効化
 7. IAM Access Analyzer を有効化
 8. S3 のデフォルト暗号化をオンに
 9. EBS のデフォルト暗号化をオンに
 10. 全リージョン向けの SCP（使うのは東京・大阪だけなら他はブロック）
- ここまでやっておけば、大事故はほぼ防げる。あとは 月 1 回の棚卸し を習慣にする。

11 IaC と運用 Tips

AWS のリソースをコンソールで作ると、再現性が低く・構成変更の履歴が追えない という問題が起きる。これを解決するのが Infrastructure as Code (IaC) である。本章では主要 IaC ツールを比較し、最後に運用で知っておきたい Tips と事故回避策をまとめる。

11.1 なぜ IaC が必要か

- ・ 再現性：同じ構成を別リージョン・別アカウントに何度でも作れる
- ・ 履歴管理：Git で変更を追える。レビューできる
- ・ 自動化：CI/CD でデプロイできる
- ・ ドキュメント：コード自体が構成の正となる
- ・ 破壊と再構築：環境を丸ごと作り直せる

個人検証でも IaC を使う最大のメリットは「**まとめて削除**」が簡単な点。コンソールで作ると消し忘れるリソースも、IaC なら `destroy` 一発で片付く。

11.2 CloudFormation

AWS 純正の IaC。YAML / JSON でリソースを宣言する。

11.2.1 基本概念

- ・ テンプレート：リソース定義の YAML / JSON
- ・ スタック：テンプレートを適用して作られたリソース群
- ・ スタックセット：複数アカウント・リージョンに展開するスタック
- ・ 変更セット：適用前の差分プレビュー

11.2.2 最小テンプレート例

```
AWSTemplateFormatVersion: '2010-09-09'
Description: A single S3 bucket

Parameters:
  BucketName:
    Type: String

Resources:
  MyBucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: !Ref BucketName
      VersioningConfiguration:
        Status: Enabled
      PublicAccessBlockConfiguration:
        BlockPublicAcls: true
```



```

    BlockPublicPolicy: true
    IgnorePublicAcls: true
    RestrictPublicBuckets: true

Outputs:
  BucketArn:
    Value: !GetAtt MyBucket.Arn

```

11.2.3 適用

```

aws cloudformation create-stack \
  --stack-name my-bucket \
  --template-body file://bucket.yaml \
  --parameters ParameterKey=BucketName,ParameterValue=my-bucket-20260421

# 更新
aws cloudformation update-stack \
  --stack-name my-bucket \
  --template-body file://bucket.yaml \
  --parameters ParameterKey=BucketName,ParameterValue=my-bucket-20260421

# 削除
aws cloudformation delete-stack --stack-name my-bucket

```

11.2.4 特徴

- AWS 純正なので新サービス対応が早い
- IAM 権限がそのまま効く
- ドリフト検知（実リソースとテンプレートの差分）
- ネイティブ機能としてロールバック
- 反面、YAML の記述量が多くなりがち

11.2.5 SAM (Serverless Application Model)

Lambda・API Gateway・DynamoDB を **短い記述** で書ける CloudFormation の拡張。サーバーレス構成に特化。

```

Transform: AWS::Serverless-2016-10-31
Resources:
  HelloApi:
    Type: AWS::Serverless::Function
    Properties:
      Runtime: python3.12
      Handler: app.handler
      CodeUri: ./src
      Events:
        Api:

```

```
Type: HttpApi
Properties:
  Path: /hello
  Method: get
```

11.3 AWS CDK (Cloud Development Kit)

CDK は、TypeScript / Python / Java / Go / C# などの **プログラミング言語** でクラウド構成を書くツール。裏では CloudFormation テンプレートを生成する。

11.3.1 特徴

- 条件分岐・ループ・ヘルパー関数などを自然に書ける
- **Construct** による抽象化（「Web サービス式」を 1 行で書く、のような）
- 型補完が効く
- エコシステム（L2 / L3 Construct、Community Constructs）が豊富

11.3.2 TypeScript での例

```
import { Stack, StackProps, RemovalPolicy } from 'aws-cdk-lib';
import { Bucket, BucketEncryption, BlockPublicAccess } from 'aws-cdk-lib/aws-s3';
import { Construct } from 'constructs';

export class MyStack extends Stack {
  constructor(scope: Construct, id: string, props?: StackProps) {
    super(scope, id, props);

    new Bucket(this, 'MyBucket', {
      versioned: true,
      encryption: BucketEncryption.S3_MANAGED,
      blockPublicAccess: BlockPublicAccess.BLOCK_ALL,
      removalPolicy: RemovalPolicy.DESTROY,
    });
  }
}
```

11.3.3 デプロイフロー

```
cdk init app --language typescript
cdk bootstrap           # 初回のみ、CDK 用の土台を作る
cdk synth               # CloudFormation YAML を生成
cdk diff                # 差分プレビュー
cdk deploy
cdk destroy
```

11.3.4 CloudFormation との使い分け

- プログラマビリティが欲しい、テストを書きたい → CDK
- YAML で宣言的に書きたい、純 AWS で揃えたい → CloudFormation / SAM

新規プロジェクトなら CDK を第一候補に挙げる開発者が増えている。

11.4 Terraform

HashiCorp が提供する **マルチクラウド対応** の IaC ツール。AWS に限定されないため、オンプレや他クラウドと混在する環境で強い。

11.4.1 特徴

- HCL (HashiCorp Configuration Language) で記述
- **プロバイダ** を差し替えれば AWS / GCP / Azure / Kubernetes / SaaS もまとめて扱える
- **state ファイル** が現実のリソースとコードの対応表になる (S3 + DynamoDB でリモート保存)
- OSS 版と HCP Terraform (旧 Terraform Cloud) あり
- ライセンス変更後のフォークとして **OpenTofu** も選択肢

11.4.2 例

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
  backend "s3" {
    bucket         = "tfstate-20260421"
    key            = "my-app/terraform.tfstate"
    region        = "ap-northeast-1"
    dynamodb_table = "tfstate-lock"
    encrypt        = true
  }
}

provider "aws" {
  region = "ap-northeast-1"
}

resource "aws_s3_bucket" "app" {
  bucket = "my-bucket-20260421"
}

resource "aws_s3_bucket_versioning" "app" {
  bucket = aws_s3_bucket.app.id
  versioning_configuration {
```

```

    status = "Enabled"
  }
}

```

11.4.3 コマンド

```

terraform init    # プロバイダと backend を初期化
terraform plan    # 差分確認
terraform apply   # 適用
terraform destroy # 削除
terraform fmt     # 整形
terraform validate

```

11.4.4 state 管理の注意

- state ファイルには機密情報が含まれる（パスワードや ARN など）ので、S3 + DynamoDB のリモート backend を使う
- state の手編集は避ける（`terraform import` / `state rm` を使う）
- 複数人・CI で運用するなら 排他ロック（DynamoDB）は必須

11.5 IaC ツール選びのガイドライン

状況	おすすめ
AWS 純度重視・小規模	CloudFormation / SAM
サーバーレスだけを素早く	SAM または CDK
プログラマビリティ重視・型安全に書きたい	CDK
マルチクラウド・オンプレ混在	Terraform / OpenTofu
既存チームが Terraform に慣れている	Terraform
K8s マニフェストも同一ツールで	Terraform + Helm プロバイダ or Pulumi

どれか1つに決める必要はなく、チームと用途に合わせて併用するのが現実解。CDKでAWS基盤を作り、Kubernetes上のアプリはHelmで運用、といった構成はよくある。

11.6 運用 Tips とハマりどころ

11.6.1 リージョン設定の一貫性

コンソールの右上リージョンはブラウザのタブごとに独立する。スタックを削除したつもりで別リージョンを見ていた、という事故がよくある。IaCを使えばリージョンは明示的にコードで指定されるため、この種の事故が防げる。

11.6.2 名前衝突の回避

- S3 バケット名、CloudFront、Route 53 レコードはグローバル一意
- IAM ロール名、Lambda 関数名、セキュリティグループ名はアカウント＋リージョン内一意
- 環境サフィックス（`-prod`、`-dev`）を入れる／乱数を付与する

11.6.3 削除できないリソース

- S3 バケットが空でない → 全バージョン削除が必要
- RDS に **削除保護**（Deletion Protection）がかかっている → 無効化してから
- VPC を消したいのにできない → ENI や EIP がどこかで紐づいている
- IAM ロールにサービスが紐づいたままになっている

「消すときのコスト」を IaC で下げる意味は大きい。

11.6.4 マルチアカウント設計

個人でも **検証専用の子アカウント** を作るのを推奨する。

- 実験で事故ってもメインのアカウントに影響しない
- 課金を明確に分離できる
- 検証が終わったらアカウントごと閉鎖できる

Organizations + Control Tower で楽に構築できる。

11.6.5 CI/CD での認証

GitHub Actions / GitLab CI から AWS を操作するときは、**アクセスキー**を置かずに **OIDC** で **AssumeRole** する。手順：

1. IAM で OIDC プロバイダ（`token.actions.githubusercontent.com`）を登録
2. 引き受け可能なロールを作成、信頼ポリシーで対象リポジトリを制限
3. Workflow で `aws-actions/configure-aws-credentials@v4` を使い、`role-to-assume` を指定

この方式なら、万一リポジトリが公開されても長期キーが漏れる心配がない。

11.6.6 よくある事故と回避

事故	回避策
NAT Gateway 立てっぱなしで月数千円	検証終了時に <code>destroy</code> 、または平日だけ稼働する IaC
Elastic IP 放置で課金	未使用の EIP を自動検知する Lambda を設置
S3 バケットを誤って公開	ブロックパブリックアクセス、Access Analyzer、Config ルール
ルートユーザー漏洩	MFA 必須、Organizations で使用を制限、CloudTrail で即時通知
リージョン間違いで不要リソース増殖	SCP で使えるリージョンを限定
CloudWatch Logs が無限に増える	ロググループごとに保持期間を必ず設定

事故	回避策
想定外の無料枠超過	Billing Preferences で Free Tier アラート有効化
IAM 権限不足で急な障害	IAM Access Analyzer Policy Generator で必要権限を算出

11.6.7 リソース棚卸しの習慣

月 1 回、以下を確認する。

- ・ **Cost Explorer** で前月比の急増がないか
- ・ 全リージョンで EC2 の停止インスタンス、未使用の EBS、未使用の EIP を点検
- ・ 停止した RDS（自動再起動でコストが戻る）
- ・ 放置された CloudFormation スタック や Terraform state
- ・ IAM ユーザーの未使用アクセスキー

Trusted Advisor（Basic プランでも一部利用可）を見ると、上記の多くが自動リスト化される。

11.7 さらに学ぶために

- ・ **AWS Well-Architected Framework**：運用・セキュリティ・信頼性・パフォーマンス・コスト・持続可能性の 6 本柱で AWS 構成をレビューする公式フレームワーク
- ・ **AWS Skill Builder**：公式の学習プラットフォーム。無料コースだけでもかなりの量
- ・ **AWS Certified Cloud Practitioner (CLF) / Solutions Architect Associate (SAA)**：資格勉強は体系学習の最短コース
- ・ **AWS ブログ / What's New**：新サービス・アップデートのキャッチアップ
- ・ 各サービスの **公式ドキュメント**：情報の正は常にドキュメント。ブログや本は補助

本書は「広く浅く」で全体像を掴むためのものである。実運用では、各サービスのドキュメントをあたり、Well-Architected の柱ごとにチェックしていく姿勢が長期的に効く。